TP1-EDO-Scipy-Sympy

March 16, 2020

[1]:
```
from IPython.core.display import HTML
css_file = './custom.css'
HTML(open(css_file, "r").read())
```

[1]:
```
<IPython.core.display.HTML object>
```

[2]:
```
from datetime import datetime
print('Last Updated On: ', datetime.now())
```

```
Last Updated On:  2020-02-24 10:31:59.155920
```

1 M62_TP1 EDO : calcul approché VS formel.

1.1 ## Calcul approché avec la fonction `odeint` du module `SciPy`

L'ECUE M62 n'est qu'une introduction aux méthodes numériques d'approximation d'EDO en utilisant Python comme langage commun de programmation. Nous allons étudier, programmer et tester plusieurs méthodes numériques.

Cependant toutes ces méthodes (et bien plus) se trouvent déjà dans le module `SciPy`.
Bien sûr vous devez toujours faire un peu de programmation avec Python pour les utiliser et une compréhension des fondements de la méthode numérique que vous utilisez est toujours indispensables.

Voyons sur trois exemples comment utiliser la fonction `odeint` du module `SciPy` pour approcher la solution d'abord d'une EDO ensuite d'un système d'EDO (ce qui inclut les équations différentielles d'ordre 2 ou plus).

1.1.1 Principe d'utilisation de `odeint`

Le principe d'utilisation de `odeint` (pour intégrer numériquement des équations différentielles) est le suivant: pour avoir une estimation numérique de la solution du problème de

$$\begin{cases} \mathbf{y}'(t) = \boldsymbol{\varphi}(t, \mathbf{y}(t)), \\ \mathbf{y}(t_0) = \mathbf{y}_0 \end{cases}$$

avec $\mathbf{y}(t) = (y_1(t), y_2(t), \ldots, y_n(t))$ le vecteur des fonctions recherchées, dépendant de la variable t et $\boldsymbol{\varphi} = (\varphi_1, \varphi_2, \ldots, \varphi_n)$ une fonction de forme quelconque, on donne comme argument la fonction $\boldsymbol{\varphi}$

(qui doit avoir deux paramètres, même dans le cas autonome, avec t comme deuxième paramètre), la condition initiale $\mathbf{y}_0$ et le domaine de temps qui nous intéresse (qui commence à t_0). Elle retourne un tableau `Numpy` (même si t était une liste).

Notons que la résolution de $y''(t) = F(t, y(t), y'(t))$ passera par celle du système différentiel

$$\begin{cases} y_1'(t) = y_2(t), \\ y_2'(t) = F(t, y_1(t), y_2(t)) \end{cases}$$

avec $y(t) = y_1(t)$, $y'(t) = y_1'(t) = y_2(t)$ et $\varphi_1(t, y_1(t), y_2(t)) = y_2(t)$, $\varphi_2(t, y_1(t), y_2(t)) = F(t, y_1(t), y_2(t))$.

1.1.2 Exemple de résolution approchée d'une équation différentielle d'ordre 1

En guise d'exemple, considérons le problème de Cauchy $y(0) = 1$ et une équation logistique simple de la forme

$$y'(t) = \frac{3}{2} y(t) \left(1 - \frac{y(t)}{6}\right).$$

On crée alors la fonction φ

[3]:
```
%reset -f
%matplotlib inline
from matplotlib.pylab import * # importe aussi numpy sans alias
from scipy.integrate import odeint
```

[4]:
```
# def phi(y,t):
#     return 1.5*y*(1-y/6)
# ou
phi = lambda y,t : 1.5*y*(1-y/6)
```

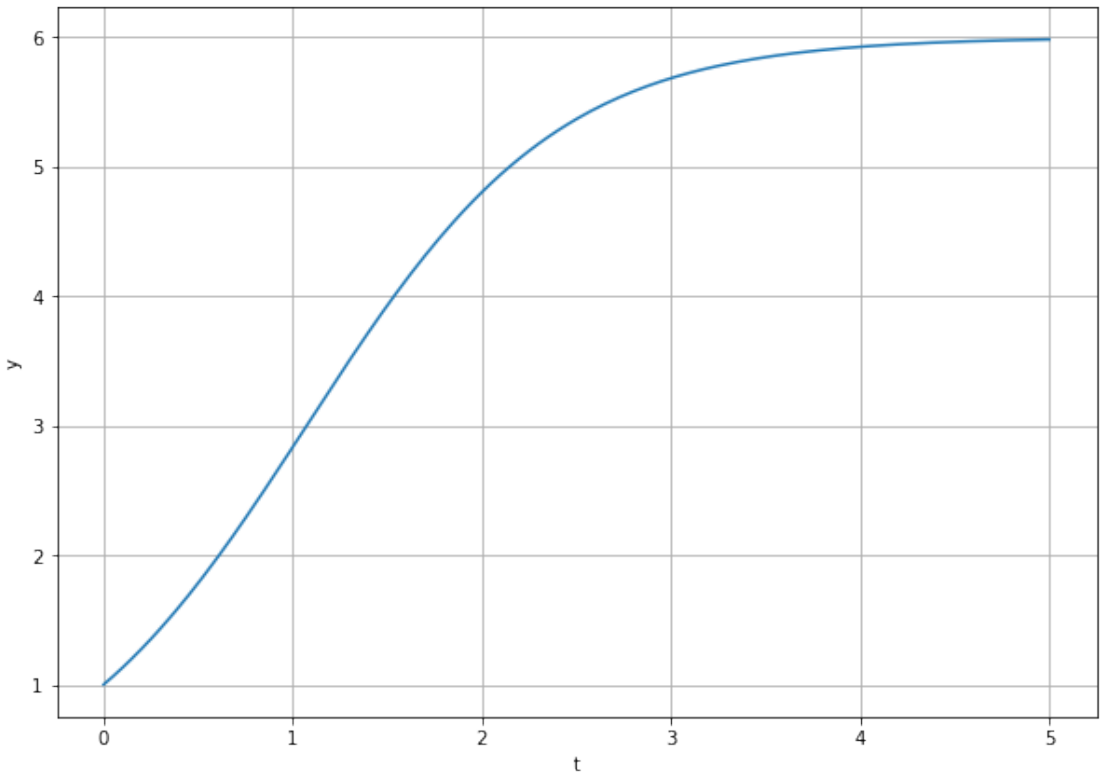
La fonction `odeint` peut être appelée avec au minimum trois arguments: - la fonction φ, - la valeur $y(t_0)$, - le vecteur t (qui commence à t_0) où la fonction y sera évaluée.

Elle renvoi un vecteur `sol` contenant l'évaluation de la solution en les points du vecteur t.

[5]:
```
t0  = 0
y0  = 1
tt  = linspace(t0,5,201)
sol = odeint(phi,y0,tt)
```

La solution peut alors être tracée simplement

[6]:
```
figure(figsize=(10,7))
plot(tt,sol)
xlabel('t')
ylabel('y')
grid();
```



NB: le nombre de points en lesquels les résultats sont évalués n'est pas (du moins directement) relié à la précision des calculs internes (ne pas imaginer que cela fixe le pas de la méthode, en particulier).

Champ de vecteurs Bien qu'il soit trés rare que l'on puisse résoudre explicitement une équation différentielle donnée, on peut souvent avoir une idée de l'allure des graphes des solutions en observant le champs de vecteurs associé. En effet le graphe d'une solution de l'équation $y'(t) = \varphi(t, y(t))$ est par définition tangent à son vecteur vitesse $(1, y'(t))$ et donc au vecteur $(1, \varphi(t, y(t))$. La connaissance de la fonction φ en chaque point (t, y) permet donc de représenter facilement ces vecteurs tangents même si l'on ne connait pas les solutions. Et si l'on en trace un grand nombre, uniformément répartis dans le plan (t, y), on obtient une représentation du champ de vecteurs associé à l'équation différentielle qui permet souvent de deviner les graphes des solutions puisqu'il s'agit des courbes qui sont tangentes en tous leurs points aux vecteurs de ce champs de vecteurs.
Il est alors intéressant d'y superposer la solution numérique obtenue avec `odeint`.

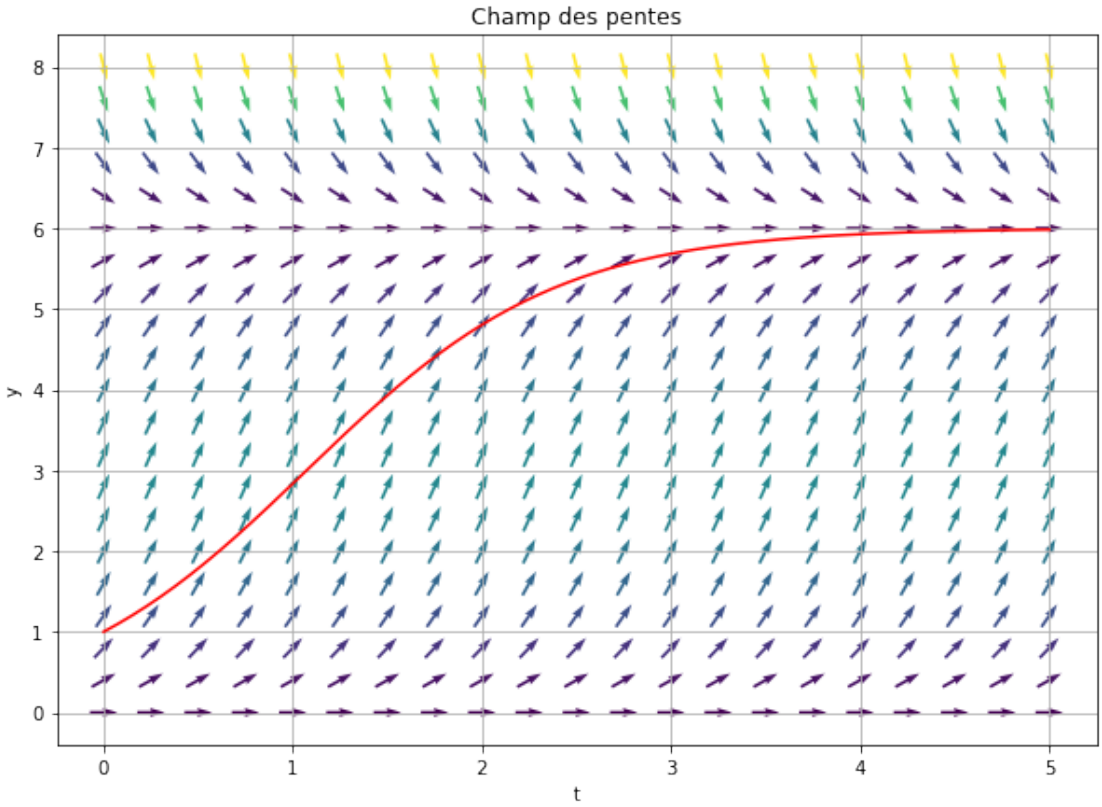
La fonction `quiver` du module `matplotlib` permet de tracer un champ de vecteurs. Utilisons-la pour obtenir celui associé à notre équation différentielle. La courbe rouge est la solution déterminée par `odeint`.

[7]:
```
figure(figsize=(10,7))

# quiverplot
```

```
# define a grid and compute direction at each point
g1  = linspace(0,5,21)
g2  = linspace(0,8,21)
T,Y = meshgrid(g1,g2)  # create a grid
DT, DY = 1, phi(Y,T)   # compute growth rate on the grid
M = sqrt(DT**2+DY**2)  # norm growth rate
M[ M==0 ] = 1          # avoid zero division errors
quiver(T,Y, DT/M, DY/M, M, pivot='mid')

plot(tt,sol,'r')
grid()
xlabel('t')
ylabel('y')
title('Champ des pentes');
```



1.1.3 Exemple de résolution approchée d'un système d'équations différentielles d'ordre 1

Considérons deux espèces: une proie (des lièvres par exemple) et un prédateur (des lynx par exemple). Ces deux populations sont représentées par $y_1(t)$ et $y_2(t)$ des fonctions continues du

temps t. Si on suppose qu'il n'y a aucune autre intervention extérieur, une modélisation possible pour ce genre de système a été proposée indépendamment par Alfred James Lotka en 1925 et Vito Volterra en 1926:

$$\begin{cases} y_1'(t) = y_1(t)(a - by_2(t)) & [\overset{\text{déf}}{=} \varphi_1(y_1(t), y_2(t), t)] \quad \text{équation équation des proies} \\ y_2'(t) = -y_2(t)(c - dy_1(t)) & [\overset{\text{déf}}{=} \varphi_2(y_1(t), y_2(t), t)] \quad \text{équation équation des prédateurs} \end{cases}$$

soit encore en notation matricielle

$$\begin{pmatrix} y_1 \\ y_2 \end{pmatrix}'(t) = \begin{pmatrix} \varphi_1(y_1(t), y_2(t), t) \\ \varphi_2(y_1(t), y_2(t), t) \end{pmatrix}.$$

On suppose qu'à ce jour il y a $y_1(0) = 2$ unités de proies (une unités = 1000 animaux) et $y_2(0) = 1$ unités de prédateurs et on se demande comment vont évoluer les populations de ces deux espèces. Pour les simulations on prendra $a = 2$, $b = 1$, $c = 1$ et $d = 0.3$.

Soit $\mathbf{y}(t) \overset{\text{déf}}{=} (y_1(t), y_2(t))$ le vecteur des deux fonctions inconnues. On commence par créer la fonction vectorielle

$$\boldsymbol{\varphi}(\mathbf{y}, t) = (\varphi_1(\mathbf{y}, t), \varphi_2(\mathbf{y}, t))$$

[8]:
```
%reset -f
%matplotlib inline
from matplotlib.pylab import * # importe aussi numpy sans alias
from scipy.integrate import odeint
```

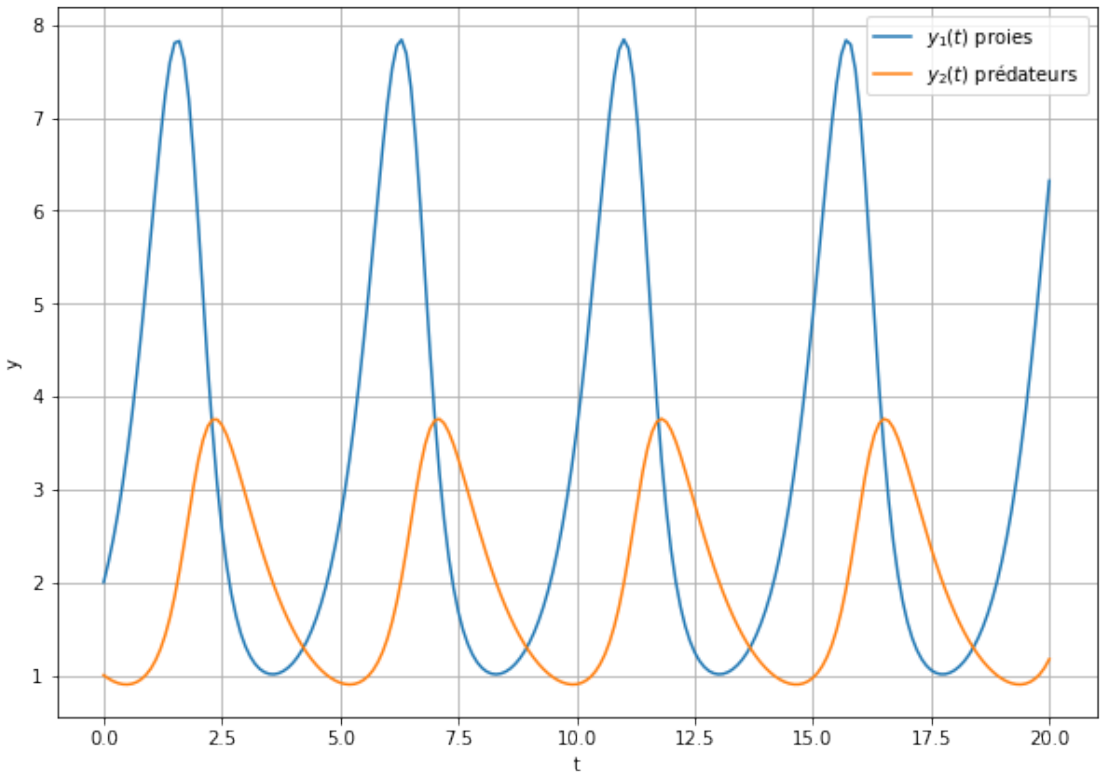
[9]:
```
# yy est une liste a deux composantes
pphi = lambda yy,t : [ yy[0]*(2-yy[1]) , -yy[1]*(1-0.3*yy[0]) ]
```

En faisant varier le temps sur l'intervalle $[0; 20]$ et en prenant comme condition initiale le vecteur $\mathbf{y}_0 = (2, 1)$ on écrit

[10]:
```
t0  = 0
yy0 = [2,1]
tt  = linspace(t0,20,201)
sol = odeint(pphi,yy0,tt)
sol_1 = sol[:,0] # [y_1(t) for t in tt]
sol_2 = sol[:,1] # [y_2(t) for t in tt]
```

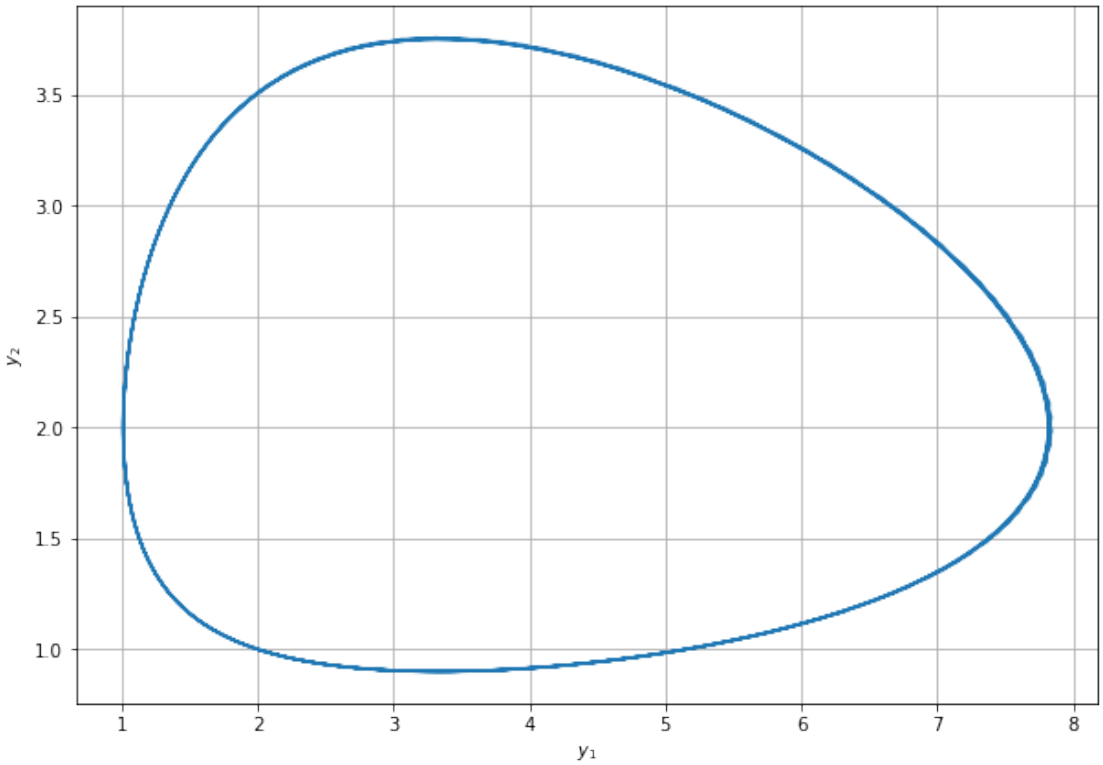
Le tracé des évolutions de y_1 et y_2 en fonction du temps t peut être obtenu par

[11]:
```
figure(figsize=(10,7))
plot(tt,sol_1,tt,sol_2)
grid()
xlabel('t')
ylabel('y')
legend([r"$y_1(t)$ proies","$y_2(t)$ prédateurs"]);
```

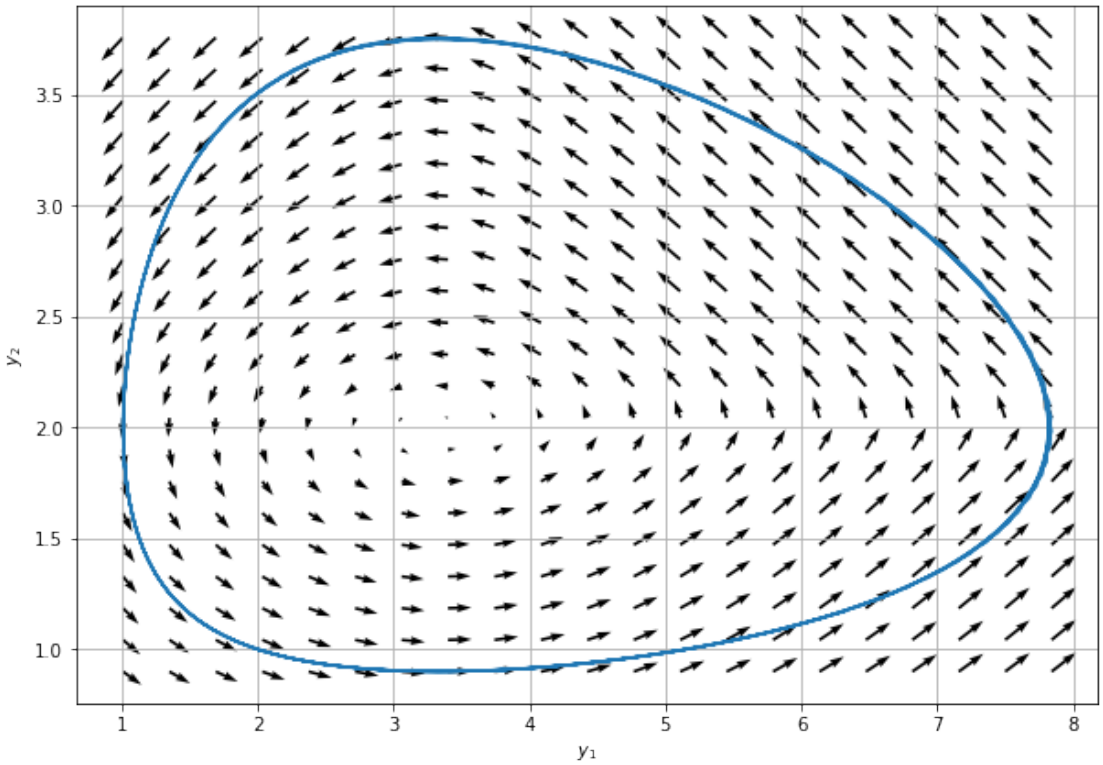


Le tracé des évolutions de y_2 en fonction de y_1 peut être obtenu par

[12]:
```
figure(figsize=(10,7))
plot(sol_1,sol_2)
grid()
xlabel(r'$y_1$')
ylabel(r'$y_2$');
```



[13]:
```
figure(figsize=(10,7))
Y1,Y2 =␣
 ↪meshgrid(linspace(min(sol_1),max(sol_1),21),linspace(min(sol_2),max(sol_2),21))
V1,V2 = pphi([Y1,Y2],tt)
r1=sqrt(1+V1**2)
r2=sqrt(1+V2**2)
quiver(Y1, Y2, V1/r1, V2/r2)
plot(sol_1,sol_2)
grid()
xlabel(r'$y_1$')
ylabel(r'$y_2$');
```



Considérons la fonction

$$E(t) = dy_1(t) + by_2(t) - c\ln(y_1(t)) - a\ln(y_2(t))$$

Vérifions analytiquement que la fonction E est une intégrale première du système, c'est-à-dire que si (y_1, y_2) est une solution alors l'application E est constante:

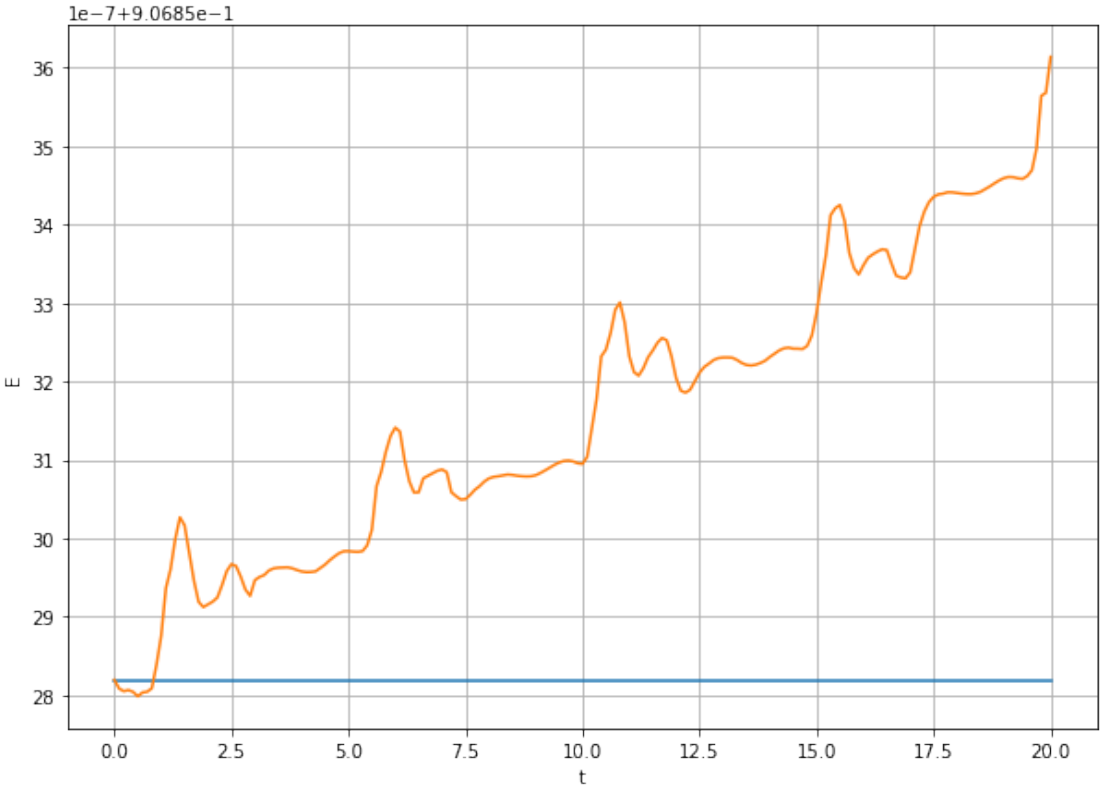
$$\begin{aligned}
E'(t) &= dy_1'(t) + by_2'(t) - \frac{c}{y_1(t)}y_1'(t) - \frac{a}{y_2(t)}y_2'(t) && (1)\\
&= \left(d - \frac{c}{y_1(t)}\right)y_1'(t) + \left(b - \frac{a}{y_2(t)}\right)y_2'(t) && (2)\\
&= \left(d - \frac{c}{y_1(t)}\right)y_1(t)(a - by_2(t)) - \left(b - \frac{a}{y_2(t)}\right)y_2(t)(c - dy_1(t)) && (3)\\
&= (dy_1(t) - c)(a - by_2(t)) - (by_2(t) - a)(c - dy_1(t)) && (4)\\
&= (dy_1(t) - c)(a - by_2(t)) - (a - by_2(t))(dy_1(t) - c) = 0. && (5)
\end{aligned}$$

Vérifions cette propriété numériquement:

[14]:
```
figure(figsize=(10,7))
# valeur pour t=0
E0 = 0.3*yy0[0]+yy0[1]-log(yy0[0])-2*log(yy0[1])
plot([tt[0],tt[-1]],[E0,E0])
```

```
EE  = 0.3*sol_1+sol_2-log(sol_1)-2*log(sol_2)
print(max(abs(EE-E0)))
plot(tt,EE)
xlabel('t')
ylabel('E')
grid();
```

```
7.94227800416e-07
```



1.1.4 Exemple de résolution approchée d'une équation différentielle d'ordre 2

Considérons l'EDO $y''(t) = -\sin(y(t))$ qui décrit le mouvement d'un pendule non amorti, équivalente au système différentiel

$$\begin{cases} y_1'(t) = y_2(t), \\ y_2'(t) = -\sin(y_1(t)) \end{cases}$$

avec $y(0) = 0$ et $y'(0) = 1$.

[15]:
```
%reset -f
%matplotlib inline
```

```
from matplotlib.pylab import * # importe aussi numpy sans alias
from scipy.integrate import odeint
```

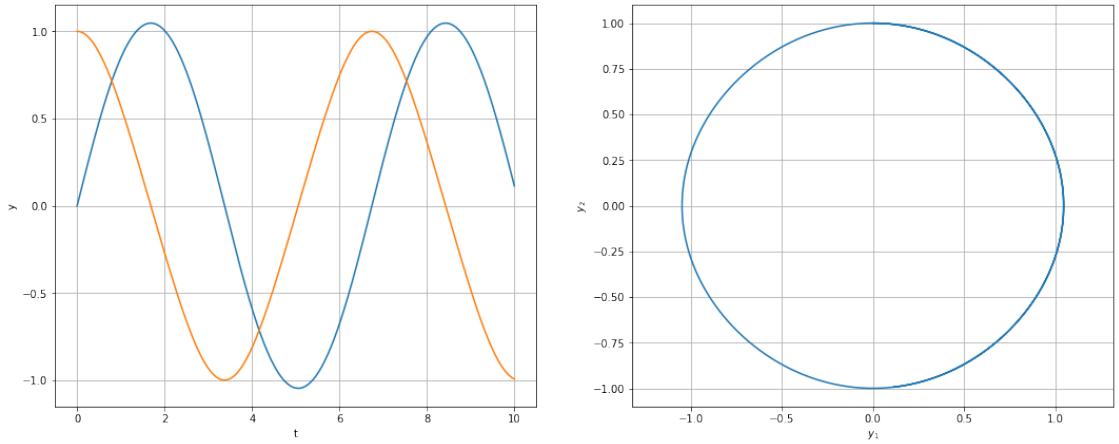
[16]:
```
pphi = lambda yy,t : [ yy[1], -sin(yy[0])  ]

t0  = 0
yy0  = [0,1]
tt  = linspace(t0,10,1001)
sol = odeint(pphi,yy0,tt)

figure(figsize=(18,7))

subplot(1,2,1)
plot(tt,sol[:,0],tt,sol[:,1])
xlabel('t')
ylabel('y')
grid()

subplot(1,2,2)
plot(sol[:,0],sol[:,1])
xlabel(r'$y_1$')
ylabel(r'$y_2$')
grid()
axis('equal');
```



Considérons l'énergie suivante (somme de l'énergie potentielle et de l'énergie cinétique):

$$E(t) = -\cos(y_1(t)) + \frac{1}{2}(y_2(t))^2$$

Vérifions analytiquement que la fonction E est conservée au cours du temps:

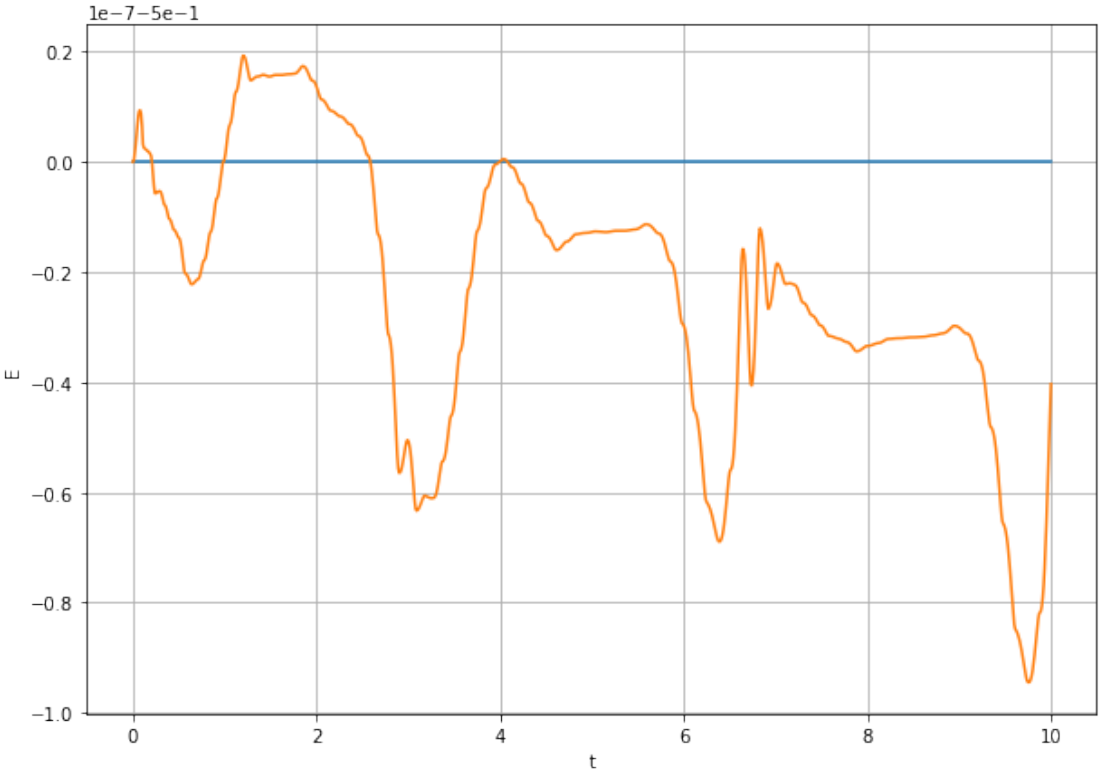
$$E'(t) = \sin(y_1(t))y_1'(t) + y_2(t)y_2'(t) \tag{6}$$
$$= -y_2'(t)y_1'(t) + y_1'(t)y_2'(t) = 0. \tag{7}$$

Vérifions cette propriété numériquement:

[17]:
```
figure(figsize=(10,7))
# valeur pour t=0
E0 = -cos(yy0[0])+yy0[1]**2/2
plot([tt[0],tt[-1]],[E0,E0])

EE  = -cos(sol[:,0])+sol[:,1]**2/2
print(max(abs(EE-E0)))
plot(tt,EE)
xlabel('t')
ylabel('E')
grid();
```

```
9.44859039897e-08
```



1.2 ## Calcul analytique avec le module sympy

https://docs.sympy.org/latest/modules/solvers/ode.html

[18]:
```
%reset -f
%matplotlib inline

import sympy as sym
sym.init_printing()
```

1.2.1 Exemple de résolution analytique d'une équation différentielle d'ordre 1

Considérons le problème de Cauchy

$$\begin{cases} u'(x) = -3x^2u(x) + 6x^2, \\ u(0) = 4. \end{cases}$$

[19]:
```
x   = sym.Symbol('x')
u   = sym.Function('u')
phi = 6*x**2-3*x**2*u(x)
edo = sym.Eq( sym.diff(u(x),x) , phi )
edo
```

[19]:

$$\frac{d}{dx}u(x) = -3x^2u(x) + 6x^2$$

[20]:
```
solgen = sym.dsolve(edo,u(x))
solgen
```

[20]:

$$u(x) = C_1e^{-x^3} + 2$$

Prise en compte des conditions initiales:

[21]:
```
x0=0
u0=4
consts = sym.solve( sym.Eq( u0, solgen.rhs.subs(x,x0)) , dict=True)[0]
consts
```

[21]:

$$\{C_1 : 2\}$$

[22]:
```
solpar=solgen.subs(consts)
solpar
```

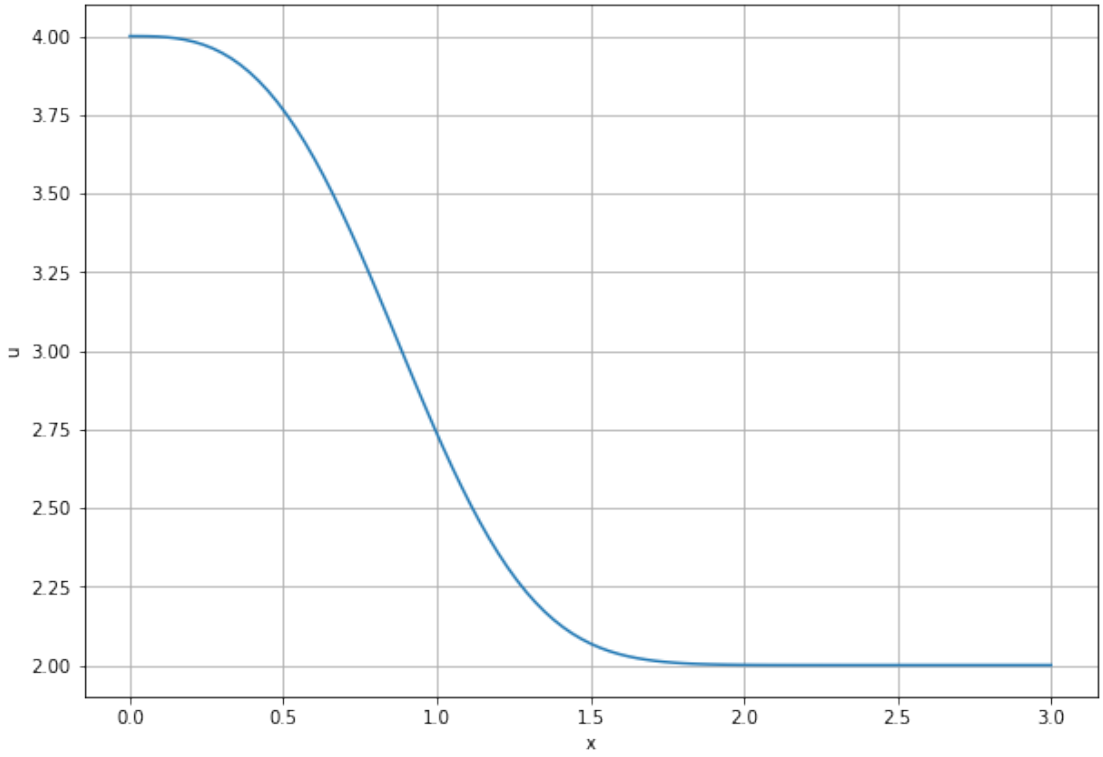
[22]:

$$u(x) = 2 + 2e^{-x^3}$$

On transforme le résultat en une fonction:

[23]:
```
func = sym.lambdify(x,solpar.rhs,'numpy')
```

[24]:
```
from matplotlib.pylab import *
figure(figsize=(10,7))
xx=linspace(0,3,101)
yy=func(xx)
plot(xx,yy)
xlabel('x')
ylabel('u')
grid();
```



1.2.2 Exemple de résolution analytique d'un système d'équations différentielles d'ordre 1

$$\begin{pmatrix} y_1 \\ y_2 \end{pmatrix}'(t) = \begin{pmatrix} \varphi_1(t, y_1(t), y_2(t)) \\ \varphi_2(t, y_1(t), y_2(t)) \end{pmatrix} = \begin{pmatrix} y_1(t) - y_2(t) \\ y_2(t) - y_1(t) \end{pmatrix}.$$

avec $t \in [0; 10]$ et en prenant comme condition initiale le vecteur $\mathbf{y}_0 = (2, 1)$.

[25]:
```
%reset -f
%matplotlib inline

import sympy as sym
sym.init_printing()

t  = sym.Symbol('t')

y1 = sym.Function('y1')
y2 = sym.Function('y2')

phi1 = y1(t)-y2(t)
phi2 = y2(t)-y1(t)

edo1 = sym.Eq( sym.diff(y1(t),t) , phi1 )
edo2 = sym.Eq( sym.diff(y2(t),t) , phi2 )
display(edo1)
display(edo2)

solgen = sym.dsolve([edo1,edo2],[y1(t),y2(t)])
display(solgen)
```

$$\frac{d}{dt}\,\mathrm{y}_1\,(t) = \mathrm{y}_1\,(t) - \mathrm{y}_2\,(t)$$

$$\frac{d}{dt}\,\mathrm{y}_2\,(t) = -\,\mathrm{y}_1\,(t) + \mathrm{y}_2\,(t)$$

$$\left[\mathrm{y}_1\,(t) = -C_1 + C_2 e^{2t}, \quad \mathrm{y}_2\,(t) = -C_1 - C_2 e^{2t}\right]$$

[26]:
```
t0  =0
y1_0=2
y2_0=1
consts = sym.solve( [sym.Eq( y1_0, solgen[0].rhs.subs(t,t0)) ,
                     sym.Eq( y2_0, solgen[1].rhs.subs(t,t0)) ] , dict=True)[0]
display(consts)

solpar_1=solgen[0].subs(consts)
solpar_2=solgen[1].subs(consts)
display(solpar_1)
display(solpar_2)

func_1 = sym.lambdify(t,solpar_1.rhs,'numpy')
func_2 = sym.lambdify(t,solpar_2.rhs,'numpy')

from matplotlib.pylab import *
```

```
figure(figsize=(18,7))

tt=linspace(0,3,101)
yy_1=func_1(tt)
yy_2=func_2(tt)

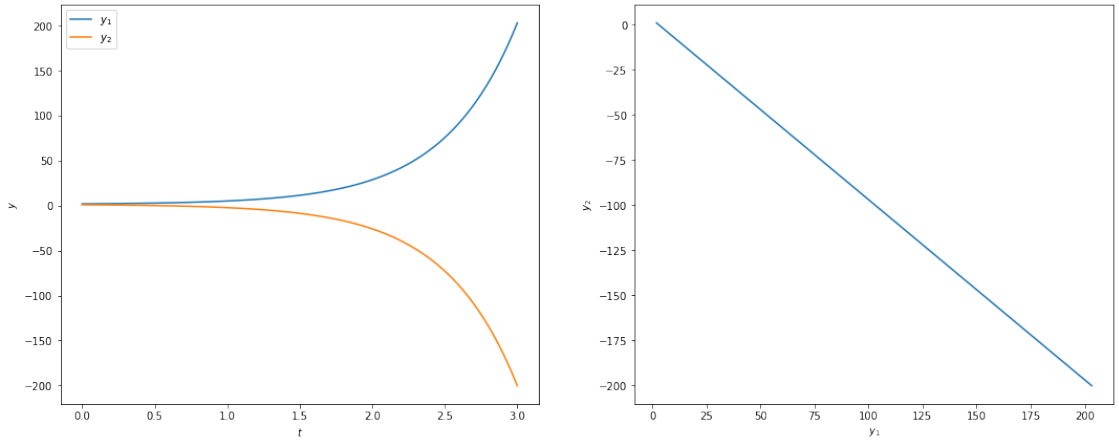
subplot(1,2,1)
plot(tt,yy_1,tt,yy_2)
legend([r'$y_1$',r'$y_2$'])
xlabel(r'$t$')
ylabel(r'$y$')

subplot(1,2,2)
plot(yy_1,yy_2)
xlabel(r'$y_1$')
ylabel(r'$y_2$');
```

$$\left\{C_1 : -\frac{3}{2}, \quad C_2 : \frac{1}{2}\right\}$$

$$\mathrm{y}_1\left(t\right) = \frac{e^{2t}}{2} + \frac{3}{2}$$

$$\mathrm{y}_2\left(t\right) = -\frac{e^{2t}}{2} + \frac{3}{2}$$



1.3 Exercices

1.3.1 Exercice (sympy)

Calculer la solution **exacte** du problème de Cauchy

$$\begin{cases} y'(t) = \frac{3}{2} y(t) \left(1 - \frac{y(t)}{6}\right), \quad t > 0 \\ y(0) = 1. \end{cases}$$

[27]:
```
%reset -f
%matplotlib inline

import sympy as sym
sym.init_printing()

t  = sym.Symbol('t')
y  = sym.Function('y')
phi= sym.Rational(3,2)*y(t)*(1-y(t)/6)
edo= sym.Eq( sym.diff(y(t),t) , phi )
display(edo)
solgen = sym.dsolve(edo,y(t))
display(solgen)
```

$$\frac{d}{dt} y(t) = \frac{3}{2} \left(-\frac{1}{6} y(t) + 1\right) y(t)$$

$$\left[y(t) = \frac{6}{e^{C_1} - e^{3t}} \left(-e^{3t} + \sqrt{e^{C_1 + 3t}}\right), \quad y(t) = -\frac{6 \left(e^{3t} + \sqrt{e^{C_1 + 3t}}\right)}{e^{C_1} - e^{3t}} \right]$$

[28]:
```
solgen=solgen[0]
display(solgen)
```

$$y(t) = \frac{6}{e^{C_1} - e^{3t}} \left(-e^{3t} + \sqrt{e^{C_1 + 3t}}\right)$$

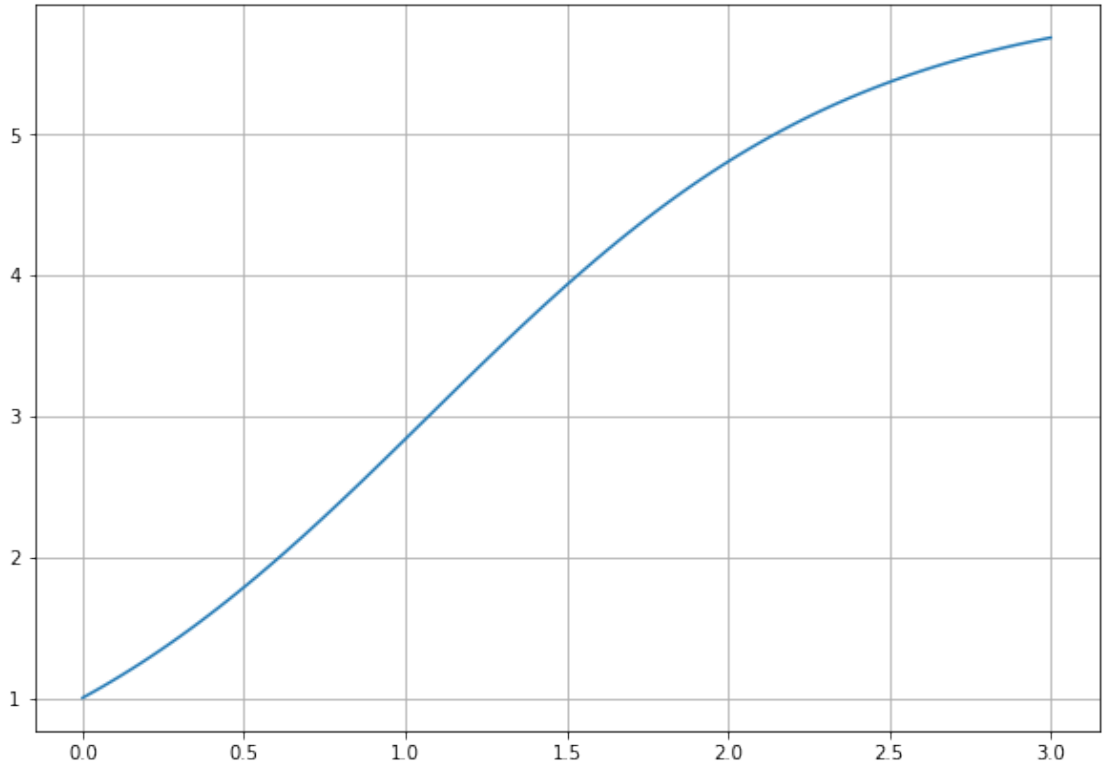
[29]:
```
t0=0
y0=1
consts = sym.solve( sym.Eq( y0, solgen.rhs.subs(t,t0)) , dict=True)[0]
display(consts)
solpar=solgen.subs(consts)
display(solpar)

func = sym.lambdify(t,solpar.rhs,'numpy')
```

```
from matplotlib.pylab import *
figure(figsize=(10,7))
tt=linspace(0,3,101)
yy=func(tt)
plot(tt,yy)
grid();
```

$$\{C_1 : \log(25)\}$$

$$y(t) = \frac{6\left(-e^{3t} + 5\sqrt{e^{3t}}\right)}{-e^{3t} + 25}$$



1.3.2 Exercice (scipy)

Calculer la solution **approchée** du problème de Cauchy

$$\begin{cases} y'(t) = \sin(ty(t)), & t > 0 \\ y(0) = a \end{cases}$$

pour $a = -2, -1, -\frac{1}{2}, -\frac{1}{10}, 0, \frac{1}{10}, \frac{1}{2}, 1, 2$ et afficher le champ de vecteurs.

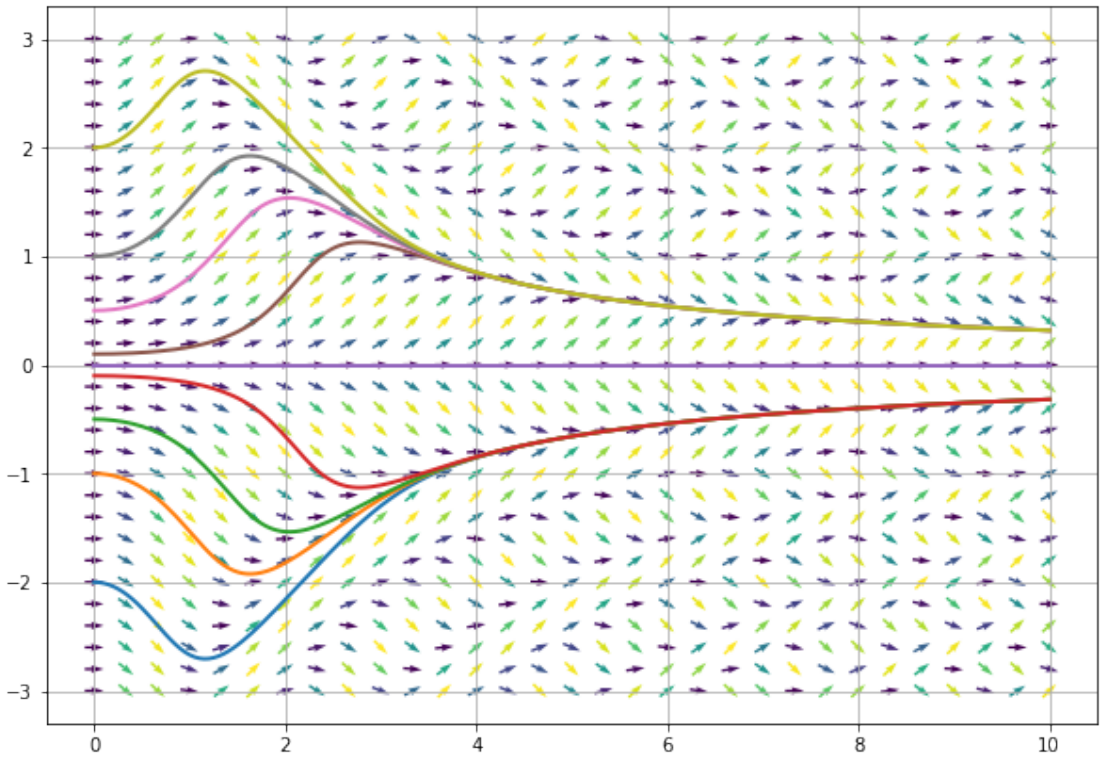
[30]:
```
%reset -f
%matplotlib inline
from matplotlib.pylab import * # importe aussi numpy sans alias
from scipy.integrate import odeint

phi = lambda y,t : sin(t*y)
tt = linspace(0,10,201)

figure(figsize=(10,7))

T,Y = meshgrid(linspace(0,10,31),linspace(-3,3,31))
DT, DY = 1, phi(Y,T)
M = sqrt(DT**2+DY**2)    # norm growth rate
M[ M == 0] = 1           # avoid zero division errors
quiver(T,Y, DT/M, DY/M, M, pivot='mid')
grid()

for y0 in [-2, -1, -0.5, -0.1, 0, 0.1, 0.5, 1, 2]:
    sol=odeint(phi,y0,tt)
    plot(tt,sol,lw=2);
```



1.3.3 Exercice (sympy - système)

1. Calculer la solution générale **exacte** du système

$$\begin{pmatrix} x(t) \\ y(t) \end{pmatrix}' = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$$

2. Pour $x(0) = -1$ et $y(0) = 1$, afficher $t \mapsto x$ et $t \mapsto y$ dans un même graphe.
3. Afficher ensuite $x \mapsto y$.
4. Calculer analytiquement $E'(t)$ avec

$$E(t) = \frac{x^2(t)}{2} + \frac{y^2(t)}{2}.$$

[31]:
```
%reset -f
%matplotlib inline

import sympy as sym
sym.init_printing()

t  = sym.Symbol('t')
x = sym.Function('x')
y = sym.Function('y')
edo1 = sym.Eq( sym.diff(x(t),t) ,  y(t) )
edo2 = sym.Eq( sym.diff(y(t),t) , -x(t) )
display(edo1)
display(edo2)
solgen = sym.dsolve([edo1,edo2],[x(t),y(t)])
display(solgen)
```

$$\frac{d}{dt}x(t) = y(t)$$

$$\frac{d}{dt}y(t) = -x(t)$$

$$[x(t) = C_1 \sin(t) + C_2 \cos(t), \quad y(t) = C_1 \cos(t) - C_2 \sin(t)]$$

[32]:
```
t_0=0
x_0=-1
y_0=1
consts = sym.solve( [ sym.Eq( x_0, solgen[0].rhs.subs(t,t_0)) , sym.Eq( y_0,␣
 ↪solgen[1].rhs.subs(t,t_0)) ] , dict=True)[0]
display(consts)
solpar_1=solgen[0].subs(consts)
solpar_2=solgen[1].subs(consts)
```

```
display(solpar_1)
display(solpar_2)

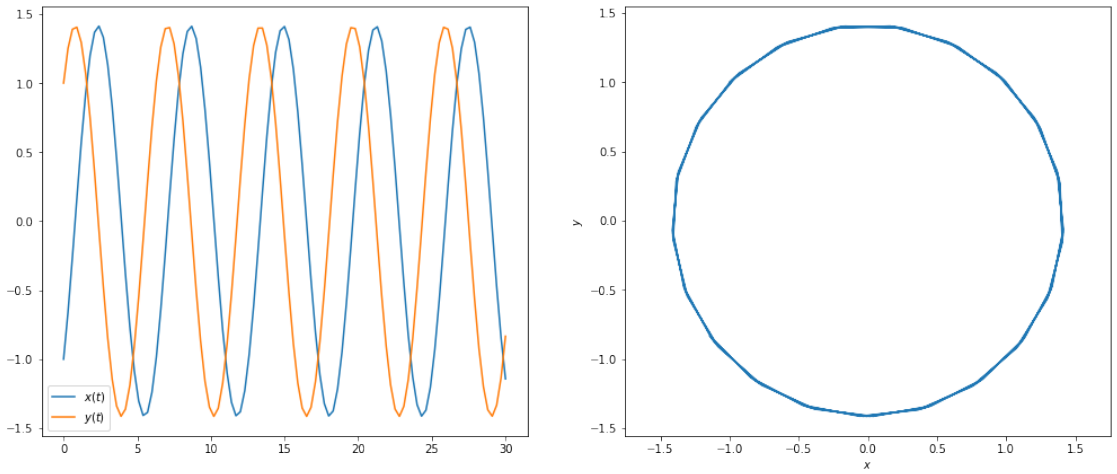
func_1 = sym.lambdify(t,solpar_1.rhs,'numpy')
func_2 = sym.lambdify(t,solpar_2.rhs,'numpy')

from matplotlib.pylab import *
figure(figsize=(17,7))
tt=linspace(0,30,101)
xx=func_1(tt)
yy=func_2(tt)
subplot(1,2,1)
plot(tt,xx,tt,yy)
legend([r'$x(t)$',r'$y(t)$'])
subplot(1,2,2)
plot(xx,yy)
xlabel(r'$x$')
ylabel(r'$y$')
axis('equal');
```

$$\{C_1 : 1, \quad C_2 : -1\}$$

$$x(t) = \sin(t) - \cos(t)$$

$$y(t) = \sin(t) + \cos(t)$$



[33]:
```
E=(solpar_1.rhs)**2/2+(solpar_2.rhs)**2/2
display(E)
```

```
Ep=sym.diff(E,t).simplify()
display(Ep)
```

$$\frac{1}{2}\left(\sin\left(t\right)-\cos\left(t\right)\right)^{2}+\frac{1}{2}\left(\sin\left(t\right)+\cos\left(t\right)\right)^{2}$$

$$0$$

1.3.4 Exercice (scipy - système)

1. Calculer la solution particulière **approchée** du système

$$\begin{cases} x'(t) = -y(t) + y^2(t)x(t) \\ y'(t) = x(t) - x^2(t)y(t) \end{cases}$$

   avec $x(0) = 0$ et $y(0) = 1$.
2. Afficher $t \mapsto x$ et $t \mapsto y$ dans un même graphe pour $t \in [0, 14]$. Afficher ensuite $x \mapsto y$.
3. Afficher

$$E(t) = \frac{x^2(t)}{2} + \frac{y^2(t)}{2}.$$

   Calculer analytiquement $E'(t)$.

[34]:
```
%reset -f
%matplotlib inline
from matplotlib.pylab import * # importe aussi numpy sans alias
from scipy.integrate import odeint

pphi = lambda yy,t : [ -yy[1]+(yy[1])**2*yy[0] , yy[0]-(yy[0])**2*yy[1] ]

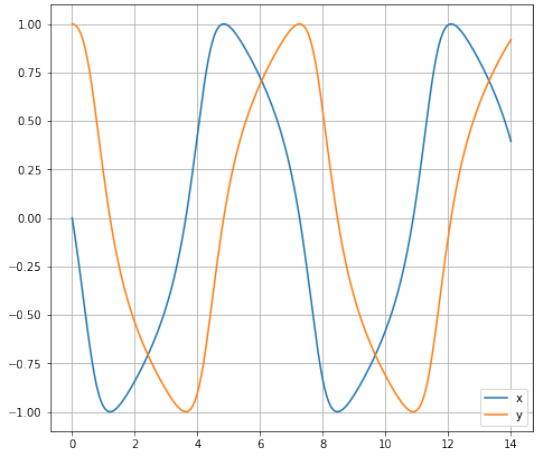
yy0  = [0,1]
tt   = linspace(0,14,201)
sol = odeint(pphi,yy0,tt)
solx,soly=sol[:,0],sol[:,1]

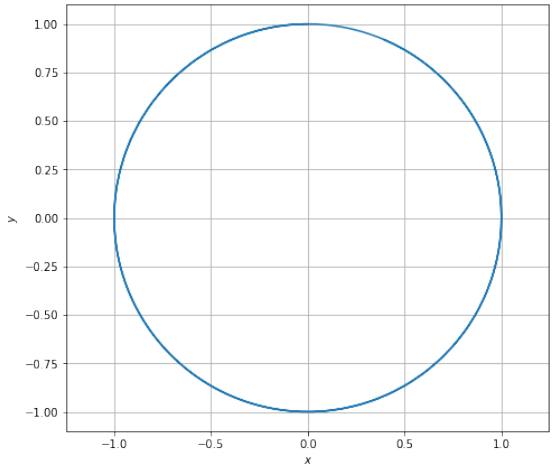
figure(figsize=(17,7))

subplot(1,2,1)
plot(tt,solx,tt,soly)
grid()
legend(["x","y"]);

subplot(1,2,2)
plot(solx,soly)
grid()
xlabel(r'$x$')
```

```
ylabel(r'$y$')
axis('equal');
```



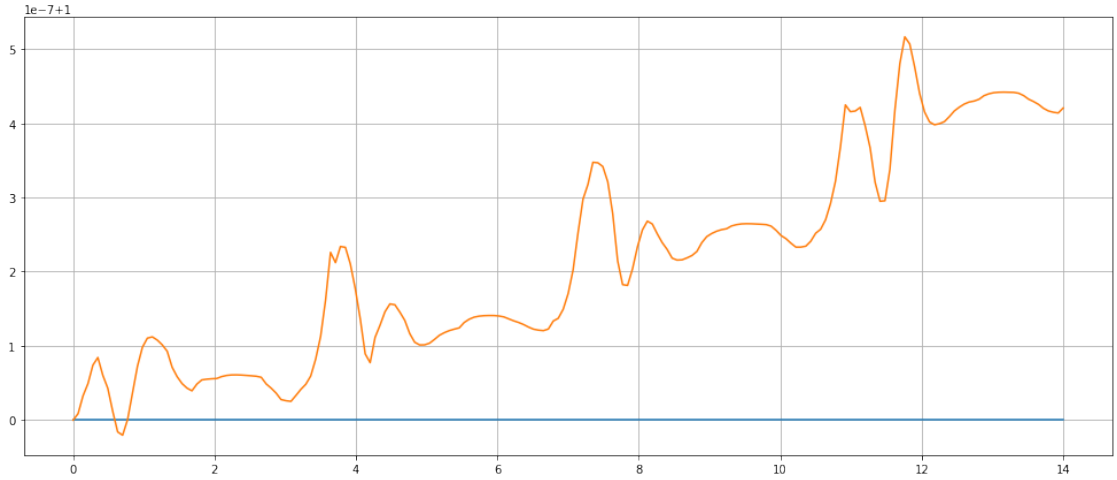


$$E'(t) = x'(t)x(t) + y'(t)y(t) = 0.$$

[35]:
```
figure(figsize=(17,7))

E0 = yy0[0]**2+yy0[1]**2
EE = solx**2+soly**2
print(max(abs(E0-EE)))
plot([tt[0],tt[-1]],[E0,E0])
plot(tt,EE)
grid();
```

```
5.16411889473e-07
```



1.3.5 Exercice (scipy - système - stabilité)

Considérons le système

$$\begin{cases} x'(t) = x^2(t) + y^2(t) - 25 \\ y'(t) = x(t)y(t) - 12 \end{cases}$$

1. Afficher le champ des vecteurs et les solutions stationnaires.
1. Calculer les solutions particulières **approchées** pour des données initiales proches des points stationnaires.

Les solutions stationnaires vérifient

$$\begin{cases} x^2 + y^2 = 25 \\ xy = 12 \end{cases}$$

On résout $x^4 - 25x^2 + 12^2 = 0$:

[36]:
```
%reset -f
%matplotlib inline

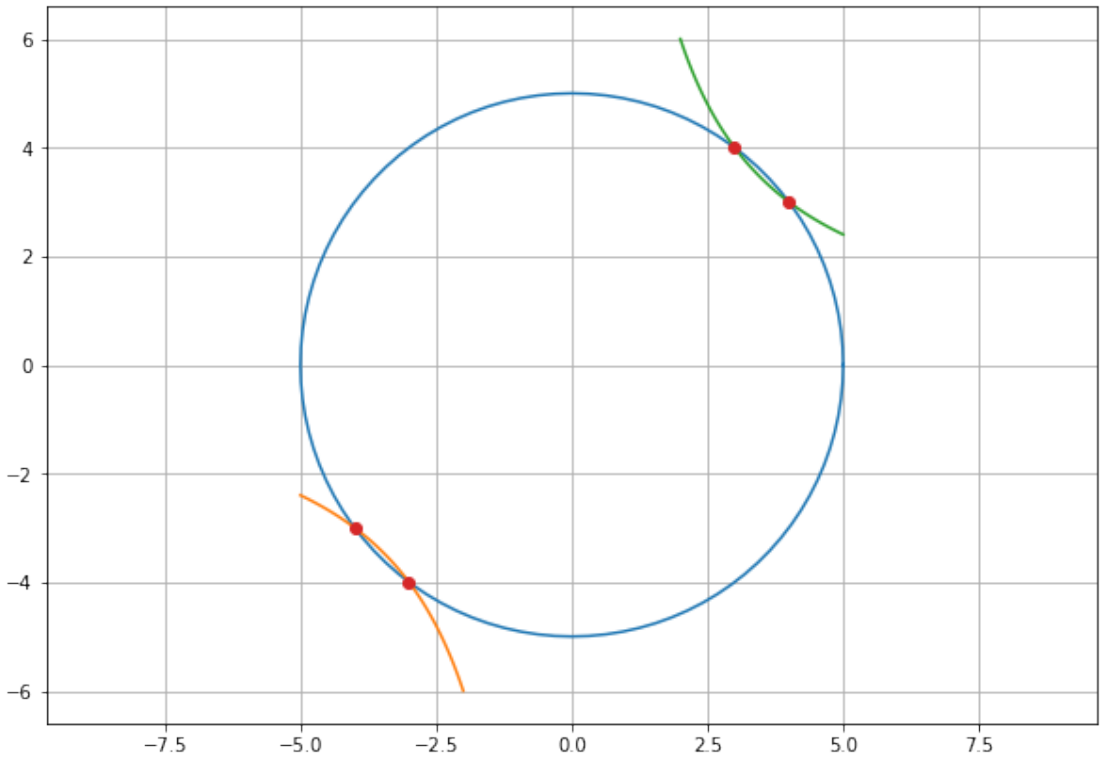
from matplotlib.pylab import * # importe aussi numpy sans alias

# numpy calcule les racines d'un polynome
x_sol=roots([1, 0, -25, 0, 12**2])
y_sol=12/x_sol
print(x_sol,y_sol)
```

```
[-4. -3.  4.  3.] [-3. -4.  3.  4.]
```

[37]:
```
figure(figsize=(10,7))

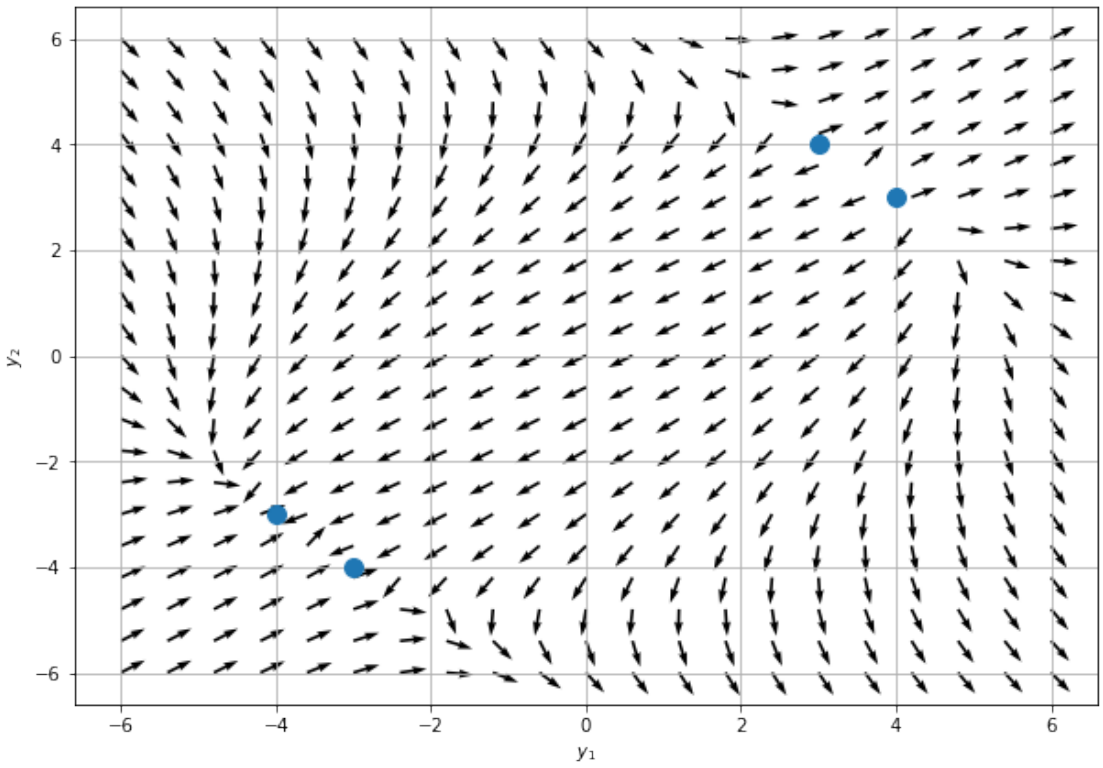
t=linspace(0,2*pi,101)
plot(5*cos(t),5*sin(t))
t=linspace(-5,-2,101)
plot(t,12/t);
t=linspace(2,5,101)
plot(t,12/t)
grid()
plot(x_sol,y_sol,'o')
axis('equal');
```



[38]:
```
from scipy.integrate import odeint


pphi = lambda y,t : [ (y[0])**2+(y[1])**2-25 , y[0]*y[1]-12 ]

figure(figsize=(10,7))
Y1,Y2 = meshgrid(linspace(-6,6,21),linspace(-6,6,21))
tt    = linspace(0,0.2,201)
V1,V2 = pphi([Y1,Y2],tt)
r=sqrt(V1**2+V2**2)
quiver(Y1, Y2, V1/r, V2/r)
plot(x_sol,y_sol,'o',ms=10)
grid()
xlabel(r'$y_1$')
ylabel(r'$y_2$');
```



[39]:
```
figure(figsize=(17,14))

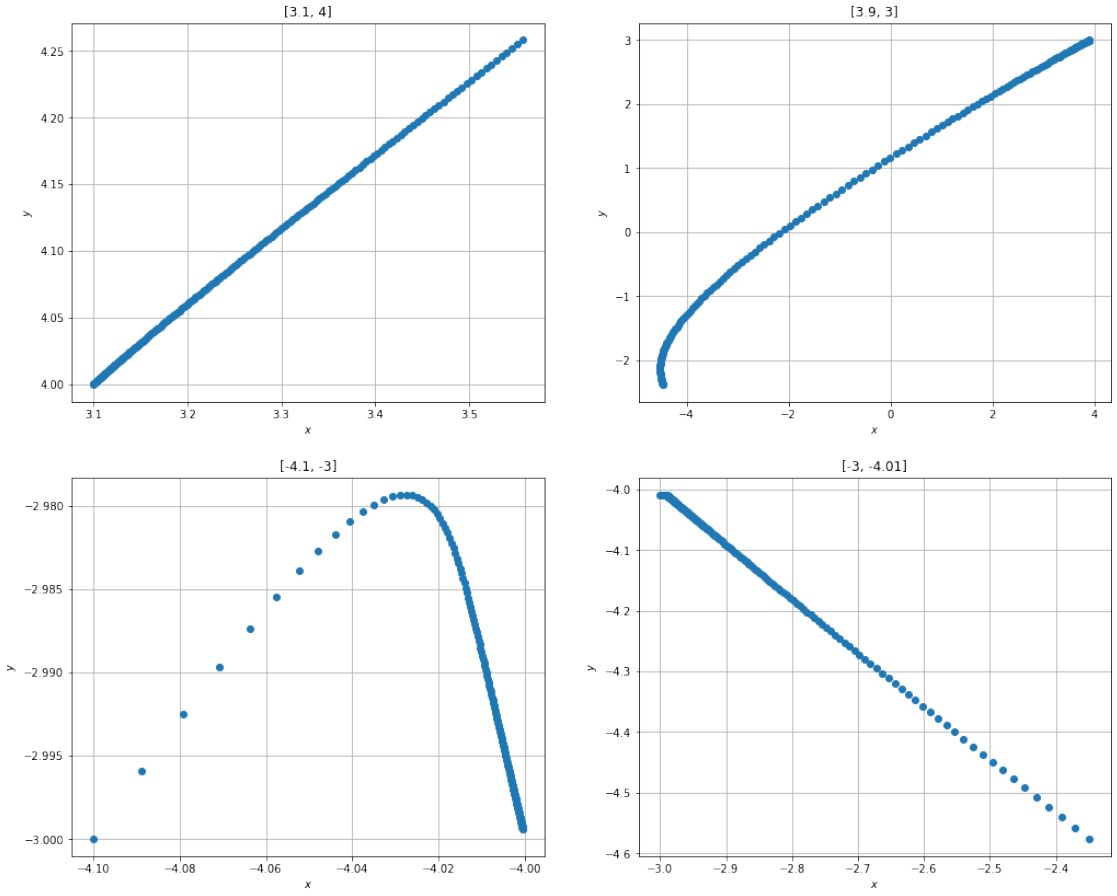
subplot(2,2,1)
y0  = [3.1,4]
tt   = linspace(0,0.2,201)
sol = odeint(pphi,y0,tt)
solx1,soly1=sol[:,0],sol[:,1]
plot(solx1,soly1,"o")
grid()
xlabel(r'$x$')
ylabel(r'$y$')
title(y0);

subplot(2,2,2)
y0  = [3.9,3]
tt   = linspace(0,1,201)
sol = odeint(pphi,y0,tt)
solx2,soly2=sol[:,0],sol[:,1]
plot(solx2,soly2,"o")
grid()
xlabel(r'$x$')
ylabel(r'$y$')
```

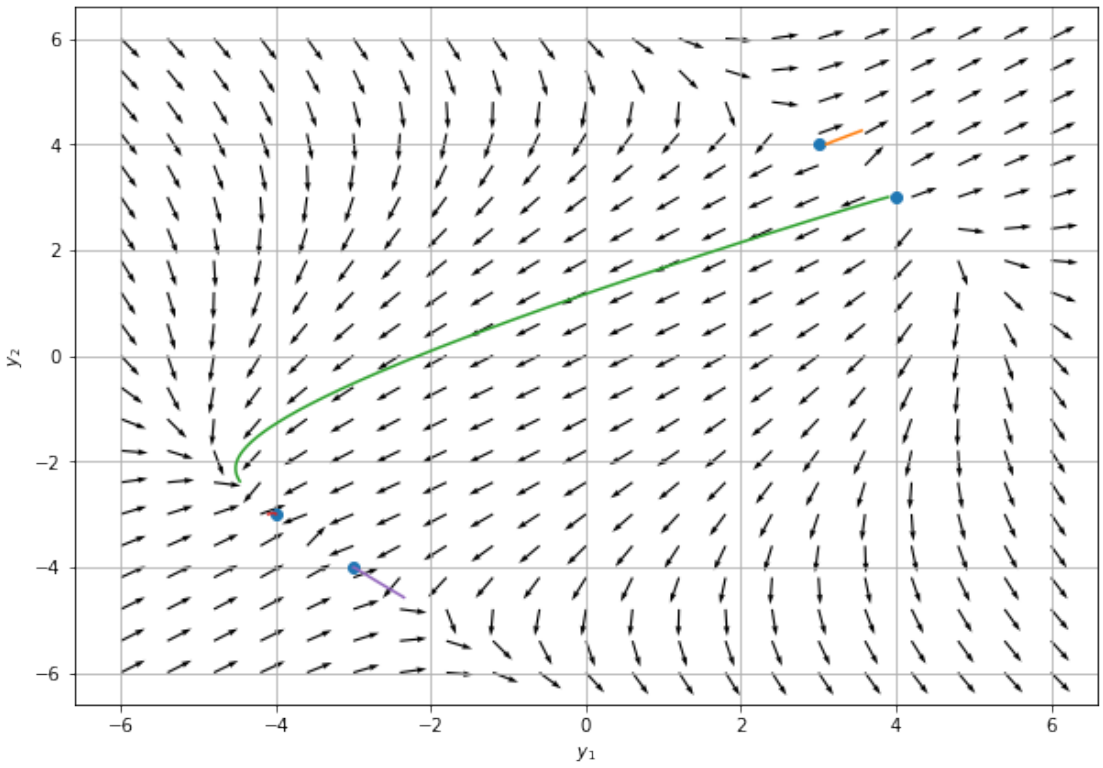
```
title(y0);

subplot(2,2,3)
y0  = [-4.1,-3]
tt   = linspace(0,3,201)
sol = odeint(pphi,y0,tt)
solx3,soly3=sol[:,0],sol[:,1]
plot(solx3,soly3,"o")
grid()
xlabel(r'$x$')
ylabel(r'$y$')
title(y0);

subplot(2,2,4)
y0  = [-3,-4.01]
tt   = linspace(0,3,201)
sol = odeint(pphi,y0,tt)
solx4,soly4=sol[:,0],sol[:,1]
plot(solx4,soly4,"o")
grid()
xlabel(r'$x$')
ylabel(r'$y$')
title(y0);
```



[40]:
```
figure(figsize=(10,7))
Y1,Y2 = np.meshgrid(linspace(-6,6,21),linspace(-6,6,21))
V1,V2 = pphi([Y1,Y2],tt)
r=sqrt(V1**2+V2**2)
quiver(Y1, Y2, V1/r, V2/r,linewidth=1,width=0.002)
plot([-3,3,-4,4],[-4,4,-3,3],'o')
grid()
xlabel(r'$y_1$')
ylabel(r'$y_2$')
plot(solx1,soly1)
plot(solx2,soly2)
plot(solx3,soly3)
plot(solx4,soly4);
```


6
4
2
0
−2
−4
−6
y_2
−6
−4
−2
0
2
4
6
y_1

[ ]:

TP2-classiques-impl-conv

March 16, 2020

[1]:
```
from IPython.display import display, Latex
from IPython.core.display import HTML
css_file = './custom.css'
HTML(open(css_file, "r").read())
```

[1]:
```
<IPython.core.display.HTML object>
```

[2]:
```
import sys #only needed to determine Python version number
print('Python version ' + sys.version)
```

```
Python version 3.6.9 (default, Nov  7 2019, 10:44:02)
[GCC 8.3.0]
```

1 M62 TP 2 - Implémentation des schémas "classiques" et étude de la convergence

1.1 Rappel de CM : implémentation des schémas d'Euler explicite et implicite

Considérons le problème de Cauchy

trouver la fonction $y\colon I \subset \mathbb{R} \to \mathbb{R}$ définie sur l'intervalle $I = [0, 1]$ telle que

$$\begin{cases} y'(t) = -4y(t) + t^2, \quad \forall t \in I = [0, 1], \\ y(0) = 1 \end{cases}$$

1. Calculer la solution exacte en utilisant le module `sympy`.
2. Calculer la solution approchée obtenue avec la méthode d'**Euler explicite** avec $h = 1/N$ et $N = 8$ (pour bien visualiser les erreurs);
3. Même exercice pour la méthode d'**Euler implicite**.

Correction 1
Calculons la solution exacte en utilisant le module `sympy`:

[3]:
```
%reset -f
%matplotlib inline

import sympy as sym
sym.init_printing()
```

```
t  = sym.Symbol('t')
y  = sym.Function('y')
edo= sym.Eq( sym.diff(y(t),t) , -4*y(t)+t**2 )
display(edo)
solgen = sym.dsolve(edo)
display(solgen)

t0=0
y0=1
consts = sym.solve( sym.Eq( y0, solgen.rhs.subs(t,t0)) , dict=True)[0]
display(consts)
solpar=solgen.subs(consts).simplify()
display(solpar)
```

$$\frac{d}{dt}y(t) = t^2 - 4y(t)$$

$$y(t) = \left(C_1 + \frac{e^{4t}}{32}\left(8t^2 - 4t + 1\right)\right)e^{-4t}$$

$$\left\{C_1 : \frac{31}{32}\right\}$$

$$y(t) = \frac{t^2}{4} - \frac{t}{8} + \frac{1}{32} + \frac{31}{32}e^{-4t}$$

On définit la solution exacte à utiliser pour estimer les erreurs:

[4]:
```
sol_exacte = sym.lambdify(t,solpar.rhs,'numpy')
```

Correction 2 et 3

On commence par importer le module `matplotlib`et la fonction `fsolve` du module `scipy.optimize` pour résoudre l'équation implicite présente dans le schéma implicite:

[5]:
```
from matplotlib.pylab import *
from scipy.optimize import fsolve
```

On initialise le problème de Cauchy

[6]:
```
# variables globales
t0     = 0
tfinal = 1
y0     = 1
```

On définit l'équation différentielle : `phi` est une lambda function qui contient la fonction mathématique $\varphi(t, y) = -4y + t^2$ dépendant des variables t et y.

[7]:
```
phi = lambda t,y : -4*y+t**2
```

On écrit les schémas numériques : les valeurs $[u_0, u_1, \ldots, u_N]$ pour chaque méthode sont contenues dans le vecteur `uu`.

Schéma d'Euler progressif :

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + h\varphi(t_n, u_n) \quad n = 0, 1, 2, \ldots N-1 \end{cases}$$

[8]:
```
# ici y0 est une variable globale
def euler_progressif(phi, tt):
    h  = tt[1] - tt[0]
    uu = [y0]
    for i in range(len(tt) - 1):
        uu.append(uu[i] + h * phi(tt[i], uu[i]))
    return uu
```

Schéma d'Euler régressif :

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + h\varphi(t_{n+1}, u_{n+1}) \quad n = 0, 1, 2, \ldots N-1 \end{cases}$$

Attention : u_{n+1} est solution de l'équation $x = u_n + h\varphi(t_{n+1}, x)$, c'est-à-dire un zéro de la fonction (en générale non linéaire)

$$x \mapsto -x + u_n + h\varphi(t_{n+1}, x)$$

[9]:
```
# ici y0 est une variable globale
def euler_regressif(phi, tt):
    h = tt[1] - tt[0]
    uu = [y0]
    for i in range(len(tt) - 1):
        temp = fsolve(lambda x: -x + uu[i] + h * phi(tt[i + 1], x), uu[i])
        uu.append(temp)
    return uu
```

On introduit la discrétisation: les nœuds d'intégration $[t_0, t_1, \ldots, t_N]$ sont contenus dans le vecteur `tt`.
On a $N+1$ points espacés de $h = \frac{t_N - t_0}{N}$.

On calcule les solutions exacte et approchées en ces points:

[10]:
```
N = 8
tt=linspace(t0,tfinal,N+1)
# print(tt)

yy    = [sol_exacte(t) for t in tt]
# yy    = sol_exacte(tt)  # si vectorisée
```

```
uu_ep = euler_progressif(phi, tt)
uu_er = euler_regressif(phi, tt)
```

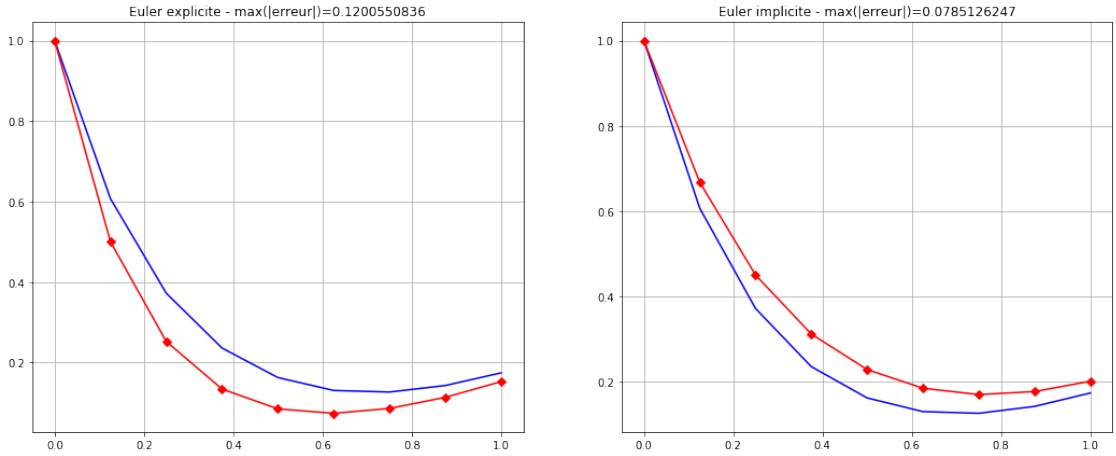
On compare les graphes des solutions exacte et approchées et on affiche le maximum de l'erreur:

$$\max_n |y(t_n) - u_n|$$

[11]:
```
figure(figsize=(18, 7))

subplot(1, 2, 1)
plot(tt, yy, 'b-', tt, uu_ep, 'r-D')
err_ep = max( [abs(uu_ep[i] - yy[i]) for i in range(N)] )
# title(f'Euler explicite - max(|erreur|)= {err_ep:1.10f}') #syntaxe si python
↪>=3.6
title('Euler explicite - max(|erreur|)=%1.10f' %(err_ep)) #"OLD" syntaxe
grid()

subplot(1, 2, 2)
plot(tt, yy, 'b-', tt, uu_er, 'r-D')
err_er = max( [abs(uu_er[i] - yy[i]) for i in range(N)] )
title('Euler implicite - max(|erreur|)=%1.10f' %(err_er)) #"OLD" syntaxe
grid();
```



1.2 Rappel de CM : étude empirique de la convergence des schémas d'Euler explicite et implicite

Considérons le même problème de Cauchy.

1. On se propose d'estimer empiriquement l'ordre de convergence de la méthode d'**Euler explicite**.

- On calcule d'abord la solution approchée avec différentes valeurs de $h_k = 1/N_k$
- Pour chaque valeur de h_k, on calcule le maximum de la valeur absolue de l'erreur et on la sauvegarde dans le vecteur `err_ep` de sort que `err_ep[k]` contient $e_k = \max_{i=0,\dots,N_k} |y(t_i) - u_i|$.
- Pour estimer l'ordre de convergence on affiche les points (`h[k]`,`err_ep[k]`) en echèlle logarithmique. On trouve ainsi une droite qui relie l'erreur au pas k à l'erreur au pas $k+1$. Pour estimer la pente globale de cette droite (par des moindres carrés) on utilisers la fonction `polyfit` avec une régression linéaire.

2. Même exercice pour estimer l'ordre de convergence de la méthode d'**Euler implicite**.

Correction

Pour chaque schéma, on calcule la solution approchée avec différentes valeurs de $h_k = 1/N_k$ et on sauvegarde les valeurs de h_k dans le vecteur `H`.

Pour chaque valeur de h_k, on calcule le maximum de la valeur absolue de l'erreur et on sauvegarde toutes ces erreurs dans le vecteur `err_schema` de sort que `err_schema[k]` contient $e_k = \max_{i=0,\dots,N_k} |y(t_i) - u_i|$.

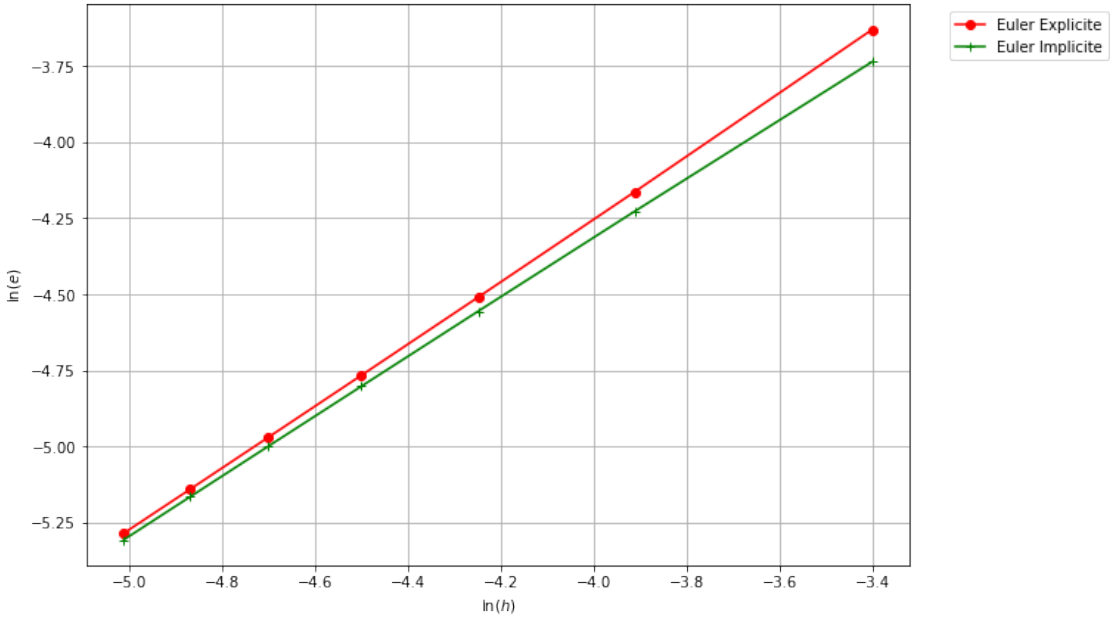
[12]:
```
H = []
err_ep = []
err_er = []

N = 10
for k in range(7):
#     N = 2**(k + 3)
    N+=20
    tt = linspace(t0, tfinal, N + 1)
    h = tt[1] - tt[0]
    yy = [sol_exacte(t) for t in tt]
    uu_ep = euler_progressif(phi, tt)
    uu_er = euler_regressif(phi, tt)
    H.append(h)
    err_ep.append(max([abs(uu_ep[i] - yy[i]) for i in range(len(yy))]))
    err_er.append(max([abs(uu_er[i] - yy[i]) for i in range(len(yy))]))
```

Pour afficher l'ordre de convergence on utilise une échelle logarithmique, i.e. on représente $\ln(h)$ sur l'axe des abscisses et $\ln(\text{err})$ sur l'axe des ordonnées.
En effet, si $\text{err} = Ch^p$ alors $\ln(\text{err}) = \ln(C) + p\ln(h)$.
En échelle logarithmique, p représente donc la pente de la ligne droite $\ln(\text{err})$.

[13]:
```
figure(figsize=(10,7))
plot(log(H), log(err_ep), 'r-o', label='Euler Explicite')
plot(log(H), log(err_er), 'g-+', label='Euler Implicite')
# loglog(H, err_ep, 'r-o', label='Euler Explicite')
# loglog(H, err_er, 'g-+', label='Euler Implicite')
xlabel('$\ln(h)$')
ylabel('$\ln(e)$')
legend(bbox_to_anchor=(1.04, 1), loc='upper left')
```

```
grid(True)
show()
```



Pour estimer l'ordre de convergence on doit estimer la pente de la droite qui relie l'erreur au pas k à l'erreur au pas $k+1$ en echelle logarithmique. Pour estimer la pente globale de cette droite (par des moindres carrés) on peut utiliser la fonction `polyfit` (du module `numpy` que nous avons déjà importé avec `matplotlib.pylab`).

[14]:
```
print ('Euler progressif %1.2f' %(polyfit(log(H),log(err_ep), 1)[0]))
print ('Euler regressif  %1.2f' %(polyfit(log(H),log(err_er), 1)[0]))
```

```
Euler progressif 1.03
Euler regressif  0.98
```

1.3 Exercice : implémentation et étude empirique de la convergence des méthodes d'Euler modifié, de Crank-Nicolson et de Heun

Ajouter l'implémentation et l'étude empirique de la convergence des méthodes d'**Euler modifié**, de **Crank-Nicolson** et de **Heun** aux exercices 1 et 2.

Correction

On écrit les trois schémas:

Schéma d'Euler modifié:

$$\begin{cases} u_0 = y_0, \\ \tilde{u} = u_n + \frac{h}{2}\varphi(t_n, u_n), \\ u_{n+1} = u_n + h\varphi\left(t_n + \frac{h}{2}, \tilde{u}\right) \end{cases} \quad n = 0, 1, 2, \dots N-1$$

qu'on peut réécrire sous la forme

$$\begin{cases} u_0 = y_0, \\ k_1 = h\varphi(t_n, u_n), \\ u_{n+1} = u_n + h\varphi\left(t_n + \frac{h}{2}, u_n + \frac{k_1}{2}\right) \end{cases} \quad n = 0, 1, 2, \dots N-1$$

[15]:
```
def euler_modifie(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        k1 = h * phi( tt[i], uu[i] )
        uu.append( uu[i]+h*phi( tt[i]+h/2 , uu[i]+k1/2 ) )
    return uu
```

Schéma de Crank-Nicolson :

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + \frac{h}{2}\Big(\varphi(t_n, u_n) + \varphi(t_{n+1}, u_{n+1})\Big) \end{cases} \quad n = 0, 1, 2, \dots N-1$$

avec u_{n+1} zéro de la fonction $x \mapsto -x + u_n + \frac{h}{2}(\varphi(t_n, u_n) + \varphi(t_{n+1}, x))$.

[16]:
```
def CN(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        temp = fsolve(lambda x: -x+uu[i]+0.5*h*(␣
 ↪phi(tt[i+1],x)+phi(tt[i],uu[i]) ), uu[i])
        uu.append(temp)
    return uu
```

Schéma de Heun:

$$\begin{cases} u_0 = y_0, \\ \tilde{u} = u_n + h\varphi(t_n, u_n) \\ u_{n+1} = u_n + \frac{h}{2}\Big(\varphi(t_n, u_n) + \varphi(t_{n+1}, \tilde{u})\Big) \end{cases} \quad n = 0, 1, 2, \dots N-1$$

qu'on peut réécrire sous la forme

$$\begin{cases} u_0 = y_0, \\ k_1 = h\varphi(t_n, u_n), \\ k_2 = h\varphi(t_{n+1}, u_n + k_1), \\ u_{n+1} = u_n + \frac{k_1+k_2}{2} \end{cases} \quad n = 0, 1, 2, \dots N-1$$

[17]:
```
def heun(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i+1], uu[i] + k1  )
        uu.append( uu[i] + (k1+k2) /2 )
    return uu
```

[18]:
```
N = 8
tt=linspace(t0,tfinal,N+1)

yy    = [sol_exacte(t) for t in tt]
uu_ep = euler_progressif(phi, tt)
uu_er = euler_regressif(phi, tt)
uu_em = euler_modifie(phi, tt)
uu_cn = CN(phi, tt)
uu_heun = heun(phi, tt)

figure(1, figsize=(18, 10))

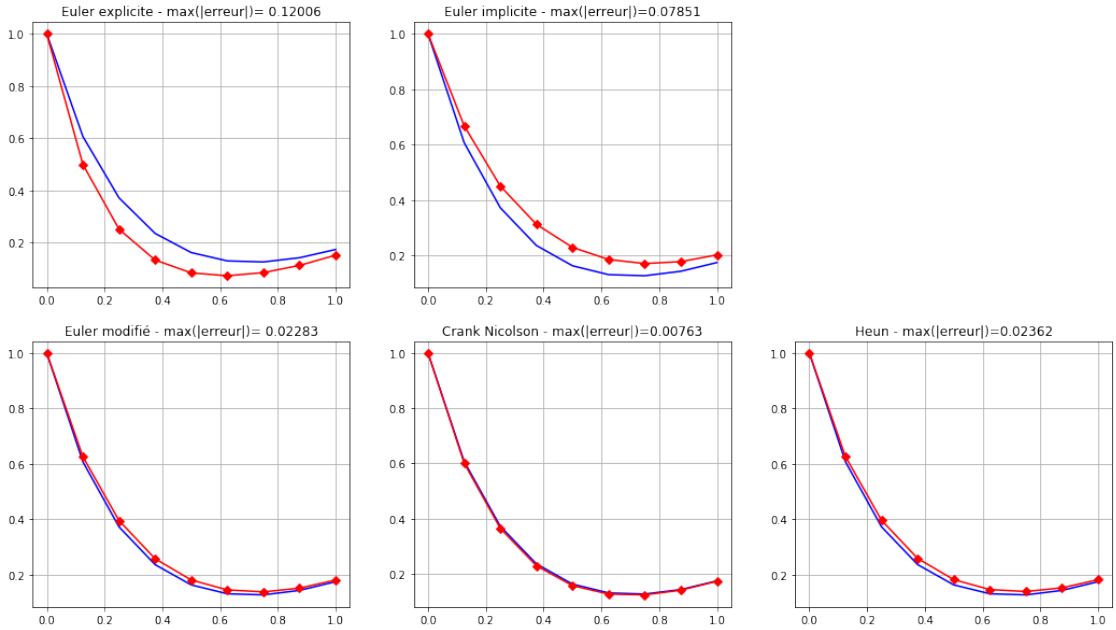
subplot(2, 3, 1)
plot(tt, yy, 'b-', tt, uu_ep, 'r-D')
err_ep = max( [abs(uu_ep[i] - yy[i]) for i in range(N)] )
# title(f'Euler explicite - max(|erreur|)= {err_ep:1.10f}')
title('Euler explicite - max(|erreur|)= %1.5f'%(err_ep))
grid()

subplot(2, 3, 2)
plot(tt, yy, 'b-', tt, uu_er, 'r-D')
err_er = max( [abs(uu_er[i] - yy[i]) for i in range(N)] )
title('Euler implicite - max(|erreur|)=%1.5f' %(err_er))
grid();


subplot(2, 3, 4)
plot(tt, yy, 'b-', tt, uu_em, 'r-D')
err_ep = max( [abs(uu_em[i] - yy[i]) for i in range(N)] )
title('Euler modifié - max(|erreur|)= %1.5f'%(err_ep))
grid()

subplot(2, 3, 5)
plot(tt, yy, 'b-', tt, uu_cn, 'r-D')
err_cn = max( [abs(uu_cn[i] - yy[i]) for i in range(N)] )
title('Crank Nicolson - max(|erreur|)=%1.5f' %(err_cn))
grid()
```

```
subplot(2, 3, 6)
plot(tt, yy, 'b-', tt, uu_heun, 'r-D')
err_heun = max( [abs(uu_heun[i] - yy[i]) for i in range(N)] )
title('Heun - max(|erreur|)=%1.5f' %(err_heun))
grid();
```



[19]:
```
H = []
err_ep = []
err_er = []
err_em = []
err_cn = []
err_heun = []

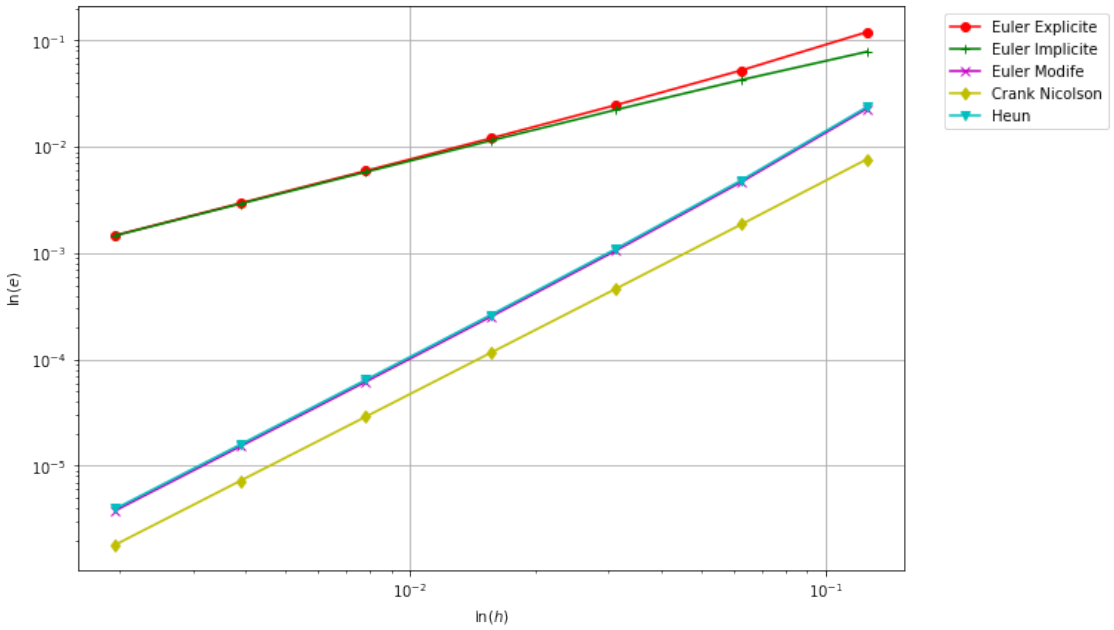
for k in range(7):
    N = 2**(k + 3)
    tt = linspace(t0, tfinal, N + 1)
    h = tt[1] - tt[0]
    yy = [sol_exacte(t) for t in tt]
    uu_ep = euler_progressif(phi, tt)
    uu_er = euler_regressif(phi, tt)
    uu_em = euler_modifie(phi, tt)
    uu_cn = CN(phi, tt)
    uu_heun = heun(phi, tt)
    H.append(h)
    err_ep.append(max([abs(uu_ep[i] - yy[i]) for i in range(len(yy))]))
    err_er.append(max([abs(uu_er[i] - yy[i]) for i in range(len(yy))]))
```

```
    err_em.append(max([abs(uu_em[i] - yy[i]) for i in range(len(yy))]))
    err_cn.append(max([abs(uu_cn[i] - yy[i]) for i in range(len(yy))]))
    err_heun.append(max([abs(uu_heun[i] - yy[i]) for i in range(len(yy))]))

figure(figsize=(10,7))
loglog(H, err_ep, 'r-o', label='Euler Explicite')
loglog(H, err_er, 'g-+', label='Euler Implicite')
loglog(H, err_em, 'm-x', label='Euler Modife')
loglog(H, err_cn, 'y-d', label='Crank Nicolson')
loglog(H, err_heun, 'c-v', label='Heun')
xlabel('$\ln(h)$')
ylabel('$\ln(e)$')
legend(bbox_to_anchor=(1.04, 1), loc='upper left')
grid(True);

print ('Euler progressif %1.2f' %(polyfit(log(H),log(err_ep), 1)[0]))
print ('Euler regressif %1.2f' %(polyfit(log(H),log(err_er), 1)[0]))
print ('Euler Modifié %1.2f' %(polyfit(log(H),log(err_em), 1)[0]))
print ('Cranck Nicolson %1.2f' %(polyfit(log(H),log(err_cn), 1)[0]))
print ('Heun %1.2f' %(polyfit(log(H),log(err_heun), 1)[0]))
```

```
Euler progressif 1.05
Euler regressif 0.96
Euler Modifié 2.08
Cranck Nicolson 2.01
Heun 2.08
```



1.4 Exercice : Fonction intégrale

Il existe des fonctions définies par des intégrales qu'on ne peut pas calculer explicitement. Il est pourtant possible de calculer des valeurs approchées de ces fonctions.

Pour $x \in \left[0; \frac{\pi}{2}\right]$ calculer et afficher la table des valeurs et tracer le graphe de la fonction

$$x \mapsto f(x) = \int_0^x \sqrt{1 - \frac{1}{4}\sin^2(t)}\mathrm{d}t$$

en approchant numériquement un problème de Cauchy (lequel?) avec la méthode d'**Euler explicite**.

Vérifier que $f\left(\frac{\pi}{6}\right) \simeq 0.51788193$ et $f\left(\frac{\pi}{2}\right) \simeq 1.46746221$.

Correction

[20]:
```
%reset -f
%matplotlib inline

from matplotlib.pylab import *
# from scipy.optimize import fsolve

# ici y0 est une variable globale qui devra être définie avant d'appeler cette␣
 ↪fonction
def euler_progressif(phi, tt):
    h  = tt[1] - tt[0]
    uu = [y0]
    for i in range(len(tt) - 1):
        uu.append(uu[i] + h * phi(tt[i], uu[i]))
    return uu
```

La fonction f est solution du problème de Cauchy

$$\begin{cases} y'(t) = \sqrt{1 - \frac{1}{4}\sin^2(t)}, \\ y(0) = 0. \end{cases}$$

En effet, $f(0) = 0$ et si on intègre l'EDO entre 0 et x on a

$$f(x) = \int_0^x \sqrt{1 - \frac{1}{4}\sin^2(t)}\mathrm{d}t = \int_0^x y'(t)\mathrm{d}t = y(x) - y(0) = y(x).$$

On définit l'équation différentielle : `phi` est une *lambda function* qui contient la fonction mathématique $\varphi(t, y)$ dépendant des variables t et y (même si y n'apparaît pas explicitement).

[21]:
```
phi = lambda t,y : sqrt(1-0.25*(sin(t))**2)
```

et la condition initiale

[22]:
```
t0, y0 = 0, 0
tfinal = pi/2
```

On introduit la discrétisation: les nœuds d'intégration $[t_0, t_1, \ldots, t_N]$ sont contenus dans le vecteur `tt`.
Comme nous devons éstimer ensuite f en $\frac{\pi}{6}$ et $\frac{\pi}{2}$, faisons en sort davoir ces deux valeurs dans nos noeuds de discrétisation.
Étant donné que - $t_0 = 0$, - $t_N = \frac{\pi}{2}$ et - $t_i = t_0 + ih$ avec $h = \frac{t_N - t_0}{N} = \frac{\pi}{2N}$,

on a $\frac{\pi}{6} = i\frac{\pi}{2N}$ ssi $i = \frac{2N}{6} = \frac{N}{3}$ et bien sûr $\frac{\pi}{2} = t_N$. Pour que $i \in \mathbb{N}$ on choisira N multiple de 3.

[23]:
```
N =12*1000
tt=linspace(t0,tfinal,N+1)
```

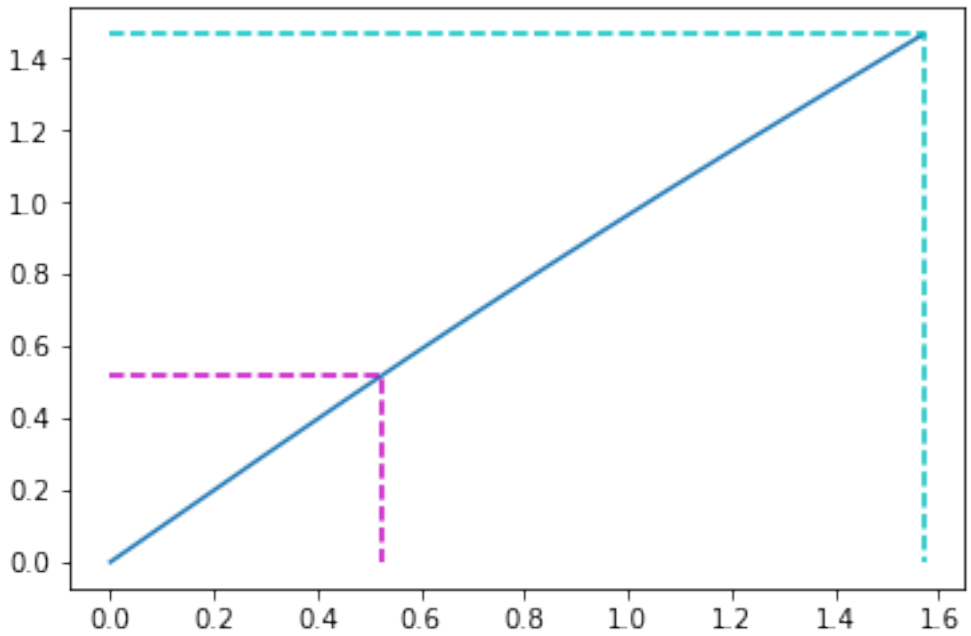
On calcule la solution approchée:

[24]:
```
uu = euler_progressif(phi,tt)
```

On vérifie que $f(\pi/6) \simeq 0.51788193$ et $f(\pi/2) \simeq 1.46746221$ et on affiche le graphe de la fonction approchée.

[25]:
```
indice=int(N/3)
# print(f"f(pi/{pi/tt[indice]})={uu[indice]}") # NEW syntaxe
# print(f"f(pi/{pi/tt[-1]})={uu[-1]}") # NEW syntaxe
print("f(pi/%g)=%g" %(pi/tt[indice],uu[indice])) # OLD syntaxe
print("f(pi/%g)=%g" %(pi/tt[-1],uu[-1])) # OLD syntaxe
plot(tt,uu)
plot([tt[0],tt[indice],tt[indice]],[uu[indice],uu[indice],0],'m--');
plot([tt[0],tt[-1],tt[-1]],[uu[-1],uu[-1],0],'c--');
```

```
f(pi/6)=0.517884
f(pi/2)=1.46747
```



Et `sympy` ?

[26]:
```
%reset -f
%matplotlib inline

import sympy as sym
sym.init_printing()

t  = sym.Symbol('t')
y  = sym.Function('y')
edo= sym.Eq( sym.diff(y(t),t) , sym.sqrt(1-sym.Rational(1,4)*(sym.sin(t))**2) )
display(edo)
# solgen = sym.dsolve(edo)
# display(solgen)

t  = sym.Symbol('t')
f=sym.Integral(sym.sqrt(1-sym.Rational(1,4)*(sym.sin(t))**2))
display(f)
# f.doit()
```

$$\frac{d}{dt}y(t) = \sqrt{-\frac{1}{4}\sin^2(t) + 1}$$

$$\int \sqrt{-\frac{1}{4}\sin^2(t) + 1}\, dt$$

1.5 Exercice : Système de deux EDO avec invariant

Considérons le problème de Cauchy

$$\begin{cases} x'(t) = -y(t), \\ y'(t) = x(t), \qquad t \in [0; 4\pi] \\ x(0) = 1, \\ y(0) = 0. \end{cases}$$

1. **Invariant**
Vérifier que $I(t) = x^2(t) + y^2(t)$ est un invariant pour le système.
1. **Approximations**
Dans une simulation numérique on aimerait que l'invariant soit préservé aussi bien que possible. Nous allons regarder ce qui se passe avec certaines méthodes vues en cours.
On notera $u_n \approx x_n = x(t_n)$ et $w_n \approx y_n = y(t_n)$.
À chaque instant t_n, on calculera $J_n \approx I_n = I(t_n)$.
Choisissons $h = t_{n+1} - t_n = \pi/48$. 1. Dans un premier temps on se propose d'appliquer la méthode d'**Euler explicite** à la résolution du système. 1. Tracer (t_n, u_n) et (t_n, w_n)

sur un même graphique; 2. sur un autre graphique tracer (t_n, J_n); 3. tracer enfin (u_n, w_n) sur un autre graphique et vérifier que la solution numérique tourne vers l'extérieur 4. Après avoir constaté que l'invariant J_n n'est pas conservé mais augment au cours du temps, montrer analytiquement que $J_n = (1+h^2)^n$. 1. Même exercice pour le schéma d'**Euler implicite** (la linéarité du second membre permet de rendre ce schéma explicite). Que peut-on constater? Commenter les résultats et écrire l'expression analytique de J_n. 1. Même exercice pour le schéma de **Crank Nicolson** (la linéarité du second membre permet de rendre ce schéma explicite). Que peut-on constater? Commenter les résultats et écrire l'expression analytique de J_n. 1. **Solution exacte** Dans ce cas il est même possible de calculer analytiquement la solution exacte. Calculer donc la solution exacte avec le module `sympy`.

Correction

Invariant

Pour tout t on a

$$I'(t) = \left(x^2(t) + y^2(t)\right)' = 2x'(t)x(t) + 2y'(t)y(t) = -2x(t)y(t) + 2x(t)y(t) = 0$$

donc $I(t) = I(0) = 0^2 + 1^2 = 1$.

Approximations

On notera $u_n \approx x_n = x(t_n)$ et $w_n \approx y_n = y(t_n)$.

À chaque instant t_n, on calculera $J_n = u_n^2 + w_n^2 \approx I_n = I(t_n)$.

Choisissons $h = t_{n+1} - t_n = \frac{\pi}{48}$.

[3]:
```
%reset -f
%matplotlib inline

from matplotlib.pylab import *
# from scipy.optimize import fsolve # non necessaire car on resout l'equation␣
 ↪implicite a la main

t0 = 0
tfinal = 4*pi

h = pi/48
tt = arange(t0,tfinal+h/2,h)

x0 = 1
y0 = 0

phi1 = lambda t,x,y : -y
phi2 = lambda t,x,y : x
```

Euler explicite

$$\begin{cases} u_0 = 1, \\ w_0 = 0, \\ u_{n+1} = u_n + h\varphi_1(t_n, u_n, w_n) = u_n - hw_n, \\ w_{n+1} = w_n + h\varphi_2(t_n, u_n, w_n) = w_n + hu_n \end{cases}$$

[4]:
```
# x0, y0, h variables globales
def euler_progressif(phi1,phi2,tt):
        uu = [x0]
        ww = [y0]
        for i in range(len(tt)-1):
                uu.append(uu[i]+h*phi1(tt[i],uu[i],ww[i]))
                ww.append(ww[i]+h*phi2(tt[i],uu[i],ww[i]))
        return [uu,ww]
```

On a

$$J_{n+1} = u_{n+1}^2 + w_{n+1}^2 = (u_n - hw_n)^2 + (w_n + hu_n)^2 = (1+h^2)(u_n^2 + w_n^2) = (1+h^2)J_n$$

soit encore

$$J_n = (1+h^2)^n J_0 = (1+h^2)^n.$$

On voit que - l'invariant n'est conservé que si $h = 0$, ce qui est impossible, - pour h fixé, $\lim_{n\to+\infty} J_n = +\infty$, - pour n fixé, l'erreur $|J_n - I_n|$ diminue comme h^2.

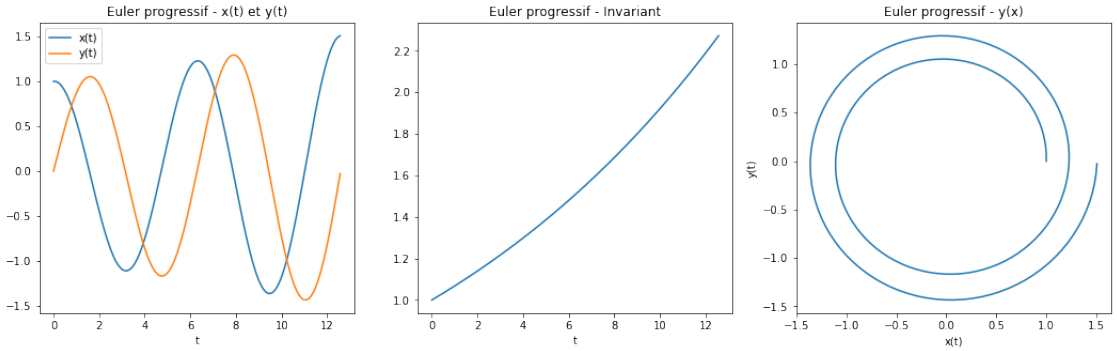
[5]:
```
[uu_ep, ww_ep] = euler_progressif(phi1,phi2,tt)
JJ_ep = [(uu_ep[i])**2+(ww_ep[i])**2 for i in range(len(tt))]


figure(figsize=(18,5))

subplot(1,3,1)
plot(tt,uu_ep,tt,ww_ep)
xlabel('t')
legend(['x(t)','y(t)'])
title('Euler progressif - x(t) et y(t)')

subplot(1,3,2)
plot(tt,JJ_ep)
xlabel('t')
title('Euler progressif - Invariant')

subplot(1,3,3)
plot(uu_ep,ww_ep)
xlabel('x(t)')
ylabel('y(t)')
title('Euler progressif - y(x)');
```



Euler implicite

$$\begin{cases} u_0 = 1, \\ w_0 = 0, \\ u_{n+1} = u_n + h\varphi_1(t_{n+1}, u_{n+1}, w_{n+1}) = u_n - hw_{n+1}, \\ w_{n+1} = w_n + h\varphi_2(t_{n+1}, u_{n+1}, w_{n+1}) = w_n + hu_{n+1} \end{cases}$$

En resolvant le système linéaire on obtient une écriture explicite

$$\begin{cases} u_0 = 1, \\ w_0 = 0, \\ u_{n+1} = \dfrac{u_n - hw_n}{1+h^2}, \\ w_{n+1} = \dfrac{w_n + hu_n}{1+h^2}. \end{cases}$$

[6]:
```
# x0,y0, h variables globales
# Notons que ici il est inutile de passer phi1,phi2 car non utilisées
 ↪explicitement dans le code
def euler_regressif(phi1,phi2,tt):
        uu = [x0]
        ww = [y0]
        for i in range(len(tt)-1):
                uu.append((uu[i]-h*ww[i])/(1+h**2))
                ww.append((ww[i]+h*uu[i])/(1+h**2))
        return [uu,ww]
```

[8]:
```
#  VERSION AVEC fsolve (non demandé)
from scipy.optimize import fsolve

# x0,y0, h variables globales
def euler_regressif(phi1,phi2,tt):
        uu = [x0]
        ww = [y0]
        for i in range(len(tt)-1):
```

```
                sys = lambda z : [ -z[0]+uu[i]+h*phi1(tt[i+1],z[0],z[1]) ,␣
↪-z[1]+ww[i]+h*phi2(tt[i+1],z[0],z[1]) ]
                utemp,wtemp = fsolve( sys , (uu[i],ww[i]) )
                uu.append(utemp)
                ww.append(wtemp)
        return [uu,ww]
```

On a

$$J_{n+1} = u_{n+1}^2 + w_{n+1}^2 = \frac{(u_n - hw_n)^2 + (w_n + hu_n)^2}{(1+h^2)^2} = (u_n^2 + w_n^2)\frac{1+h^2}{(1+h^2)^2} = \frac{1+h^2}{(1+h^2)^2} J_n = \frac{1}{1+h^2} J_n$$

soit encore

$$J_n = \left(\frac{1}{1+h^2}\right)^n J_0 = \left(\frac{1}{1+h^2}\right)^n .$$

On voit que - l'invariant n'est conservé que si $h = 0$, ce qui est impossible, - pour h fixé, $\lim_{n\to+\infty} J_n = 0^+$, - pour n fixé, l'erreur $|J_n - I_n|$ diminue comme h^2.

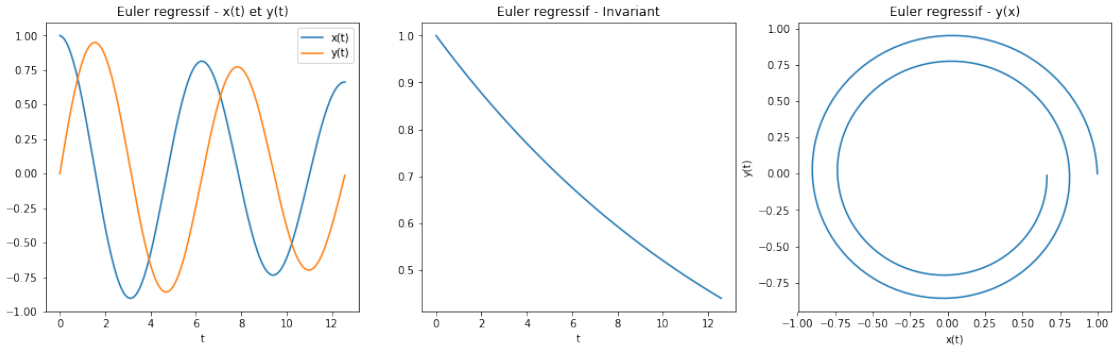
[9]:
```
[uu_er, ww_er] = euler_regressif(phi1,phi2,tt)
JJ_er = [(uu_er[i])**2+(ww_er[i])**2 for i in range(len(tt))]

figure(figsize=(18,5))

subplot(1,3,1)
plot(tt,uu_er,tt,ww_er)
xlabel('t')
legend(['x(t)','y(t)'])
title('Euler regressif - x(t) et y(t)')

subplot(1,3,2)
plot(tt,JJ_er)
xlabel('t')
title('Euler regressif - Invariant')

subplot(1,3,3)
plot(uu_er,ww_er)
xlabel('x(t)')
ylabel('y(t)')
title('Euler regressif - y(x)');
```



Crank Nicolson

$$\begin{cases} u_0 = 1, \\ w_0 = 0, \\ u_{n+1} = u_n + \frac{h}{2}\Big(\varphi_1(t_n, u_n, w_n) + \varphi_1(t_{n+1}, u_{n+1}, w_{n+1})\Big) = u_n - \frac{h}{2}w_n - \frac{h}{2}w_{n+1}, \\ w_{n+1} = w_n + \frac{h}{2}\Big(\varphi_2(t_n, u_n, w_n) + \varphi_2(t_{n+1}, u_{n+1}, w_{n+1})\Big) = w_n + \frac{h}{2}u_n + \frac{h}{2}u_{n+1}. \end{cases}$$

En resolvant le système linéaire on obtient une écriture explicite:

$$\begin{cases} u_0 = 1, \\ w_0 = 0, \\ u_{n+1} = \dfrac{(4-h^2)u_n - 4hw_n}{4+h^2}, \\ w_{n+1} = \dfrac{(4-h^2)w_n + 4hu_n}{4+h^2}. \end{cases}$$

De plus, on a

$$\begin{aligned} J_{n+1} &= u_{n+1}^2 + w_{n+1}^2 \\ &= \frac{((4-h^2)u_n - 4hw_n)^2 + ((4-h^2)w_n + 4hu_n)^2}{(4+h^2)^2} \\ &= \frac{(4-h^2)^2u_n^2 + 16h^2w_n^2 - 8(4-h^2)hu_nw_n + (4-h^2)^2w_n^2 + 16h^2u_n^2 + 8(4-h^2)hu_nw_n}{(4+h^2)^2} \\ &= \frac{\big((4-h^2)^2 + 16h^2\big)(u_n^2 + w_n^2)}{(4+h^2)^2} \\ &= \frac{(4+h^2)^2(u_n^2 + w_n^2)}{(4+h^2)^2} \\ &= (u_n^2 + w_n^2) = J_n \end{aligned}$$

On voit que l'invariant est conservé pour tout h

Vérifions ces calculs avec `sympy`:

[32]:
```
import sympy as sym
sym.init_printing()

sym.var('u_n, u_np1, w_n, w_np1, dt')
eq1 = sym.Eq( u_np1 , u_n-dt/2*w_n-dt/2*w_np1 )
eq2 = sym.Eq( w_np1 , w_n+dt/2*u_n+dt/2*u_np1 )
sol=sym.solve([eq1,eq2],[u_np1,w_np1],dict=True)[0]
display(sol)

J_np1=u_np1**2+w_np1**2
J_np1.subs(sol).simplify()
```

$$\left\{u_{np1} : \frac{1}{dt^2+4}\left(-dt^2 u_n - 4dtw_n + 4u_n\right),\quad w_{np1} : \frac{1}{dt^2+4}\left(-dt^2 w_n + 4dtu_n + 4w_n\right)\right\}$$

[32]:

$$u_n^2 + w_n^2$$

[10]:
```
# x0,y0, h variables globales
# Notons que ici il est inutile de passer phi1,phi2 car non utilisées␣
 ↪explicitement dans le code
def cn(phi1,phi2,tt):
        uu = [x0]
        ww = [y0]
        for i in range(len(tt)-1):
                uu.append(((4-h**2)*uu[i]-4*h*ww[i])/(4+h**2))
                ww.append(((4-h**2)*ww[i]+4*h*uu[i])/(4+h**2))
        return [uu,ww]
```

[12]:
```
#  VERSION AVEC fsolve (non demandé)
from scipy.optimize import fsolve

# x0,y0, h variables globales
def cn(phi1,phi2,tt):
        uu = [x0]
        ww = [y0]
        for i in range(len(tt)-1):
                sys = lambda z : [ -z[0]+uu[i]+h/
 ↪2*(phi1(tt[i],uu[i],ww[i])+phi1(tt[i+1],z[0],z[1])) ,
                                -z[1]+ww[i]+h/
 ↪2*(phi2(tt[i],uu[i],ww[i])+phi2(tt[i+1],z[0],z[1])) ]
                utemp,wtemp = fsolve( sys , (uu[i],ww[i]) )
                uu.append(utemp)
                ww.append(wtemp)
        return [uu,ww]
```

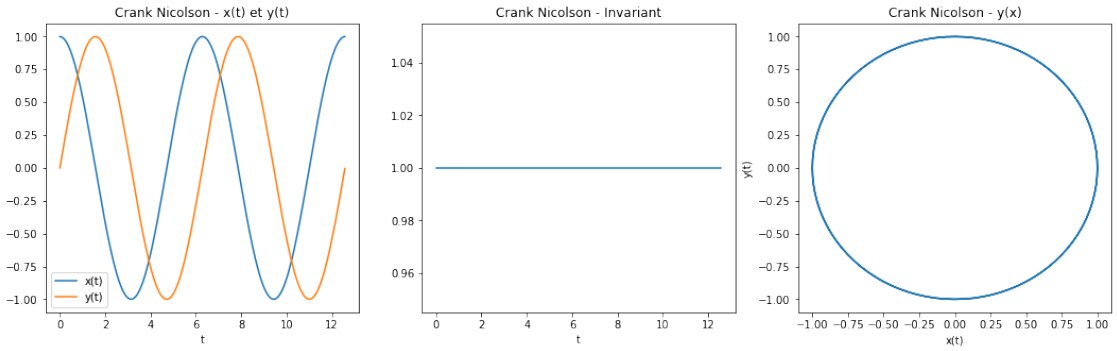
[13]:
```
[uu_cn, ww_cn] = cn(phi1,phi2,tt)
JJ_cn = [(uu_cn[i])**2+(ww_cn[i])**2 for i in range(len(tt))]

figure(figsize=(18,5))

subplot(1,3,1)
plot(tt,uu_cn,tt,ww_cn)
xlabel('t')
legend(['x(t)','y(t)'])
title('Crank Nicolson - x(t) et y(t)')

subplot(1,3,2)
plot(tt,JJ_cn)
xlabel('t')
title('Crank Nicolson - Invariant')

subplot(1,3,3)
plot(uu_cn,ww_cn)
xlabel('x(t)')
ylabel('y(t)')
title('Crank Nicolson - y(x)');
```



Solution exacte

La solution exacte est

$$x(t) = \cos(t),$$
$$y(t) = \sin(t).$$

Cette solution est périodique de période 2π et on a bien

$$I(t) = x^2(t) + y^2(t) = 1 \quad \forall t.$$

Vérifions-le avec le module sympy:

[14]:
```
%reset -f
%matplotlib inline

import sympy as sym
sym.init_printing()

t  = sym.Symbol('t')
x = sym.Function('x')
y = sym.Function('y')
edo1 = sym.Eq( sym.diff(x(t),t) , -y(t) )
edo2 = sym.Eq( sym.diff(y(t),t) ,  x(t) )
display(edo1)
display(edo2)
solgen = sym.dsolve([edo1,edo2],[x(t),y(t)])
display(solgen)

t_0=0
x_0=1
y_0=0
consts = sym.solve( [ sym.Eq( x_0, solgen[0].rhs.subs(t,t_0)) , sym.Eq( y_0,␣
 ↪solgen[1].rhs.subs(t,t_0)) ] , dict=True)[0]
display(consts)
solpar_1=solgen[0].subs(consts)
solpar_2=solgen[1].subs(consts)
display(solpar_1)
display(solpar_2)

func_1 = sym.lambdify(t,solpar_1.rhs,'numpy')
func_2 = sym.lambdify(t,solpar_2.rhs,'numpy')

from matplotlib.pylab import *
figure(figsize=(17,7))
tt=linspace(0,30,501)
xx=func_1(tt)
yy=func_2(tt)
subplot(1,2,1)
plot(tt,xx,tt,yy)
legend([r'$x(t)$',r'$y(t)$'])
subplot(1,2,2)
plot(xx,yy)
xlabel(r'$x$')
ylabel(r'$y$')
axis('equal');
```

$$\frac{d}{dt}x(t) = -y(t)$$

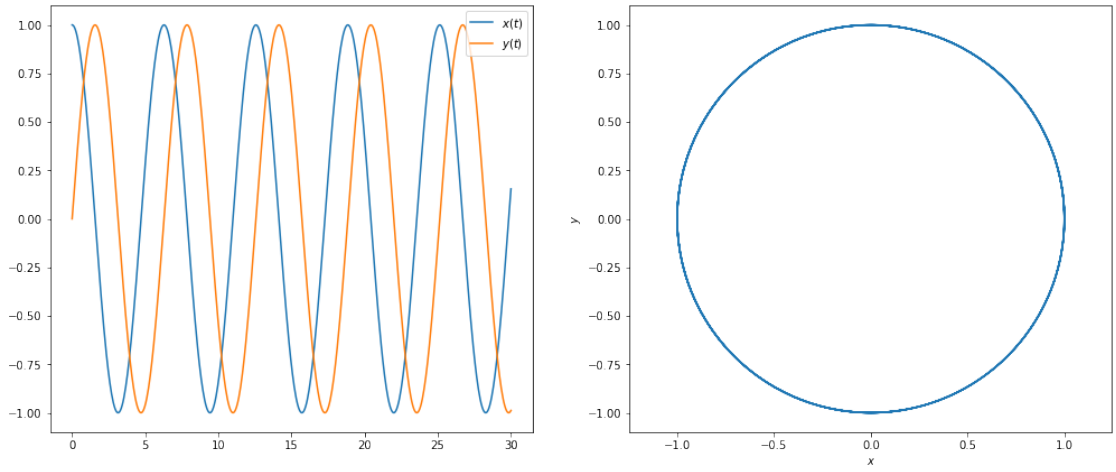
$$\frac{d}{dt}y(t) = x(t)$$

$$[x(t) = -C_1 \sin(t) - C_2 \cos(t), \quad y(t) = C_1 \cos(t) - C_2 \sin(t)]$$

$$\{C_1 : 0, \quad C_2 : -1\}$$

$$x(t) = \cos(t)$$

$$y(t) = \sin(t)$$



[ ]:

TP3-Multisteps-impl

March 16, 2020

[1]:
```
from IPython.display import display, Latex
from IPython.core.display import HTML
css_file = './custom.css'
HTML(open(css_file, "r").read())
```

[1]:
```
<IPython.core.display.HTML object>
```

[2]:
```
import sys #only needed to determine Python version number
print('Python version ' + sys.version)
```

```
Python version 3.6.9 (default, Nov  7 2019, 10:44:02)
[GCC 8.3.0]
```

1 M62 TP 3 - Implémentation de schémas

Compléter le notebook en ajoutant l'implémentation des schémas indiqués et en vérifiant l'impléméntation sur l'exemple donné.

1.1 Implémentation des schémas

On écrit les schémas numériques : + les nœuds d'intégration $[t_0, t_1, \ldots, t_N]$ sont contenus dans le vecteur `tt` + les valeurs $[u_0, u_1, \ldots, u_N]$ pour chaque méthode sont contenues dans le vecteur `uu`.

[3]:
```
%reset -f
%matplotlib inline
%autosave 300

from matplotlib.pylab import *
from scipy.optimize import fsolve
```

```
Autosaving every 300 seconds
```

1.1.1 Schémas explicites

Schéma d'Euler progressif = de Adam-Bashforth à 1 pas

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + h\varphi(t_n, u_n) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

[4]:
```
def euler_progressif(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        uu.append(uu[i]+h*phi(tt[i],uu[i]))
    return uu
```

Schéma de Adam-Bashforth à 2 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ u_{n+1} = u_n + \dfrac{h}{2}\Big(3\varphi(t_n, u_n) - \varphi(t_{n-1}, u_{n-1})\Big) \quad n = 1, 2, 3, 4, 5, \dots N-1 \end{cases}$$

[5]:
```
def AB2(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    for i in range(1,len(tt)-1):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        uu.append( uu[i] + (3*k1-k2) /2 )
    return uu
```

Schéma de Adam-Bashforth à 3 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ u_2 = u_1 + \dfrac{h}{2}\Big(3\varphi(t_1, u_1) - \varphi(t_0, u_0)\Big), \\ u_{n+1} = u_n + \dfrac{h}{12}\Big(23\varphi(t_n, u_n) - 16\varphi(t_{n-1}, u_{n-1}) + 5\varphi(t_{n-2}, u_{n-2})\Big) \quad n = 2, 3, 4, 5, \dots N-1 \end{cases}$$

[6]:
```
def AB3(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    uu.append(uu[1]+h*(3*phi(tt[1],uu[1])-phi(tt[0],uu[0]))/2)
    for i in range(2,len(tt)-1):
```

```
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        k3 = h * phi( tt[i-2], uu[i-2] )
        uu.append( uu[i] + (23*k1-16*k2+5*k3) /12 )
    return uu
```

Schéma de Adam-Bashforth à 4 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ u_2 = u_1 + \frac{h}{2}\Big(3\varphi(t_1, u_1) - \varphi(t_0, u_0)\Big), \\ u_3 = u_2 + \frac{h}{12}\Big(23\varphi(t_2, u_2) - 16\varphi(t_1, u_1) + 5\varphi(t_0, u_0)\Big), \\ u_{n+1} = u_n + \frac{h}{24}\Big(55\varphi(t_n, u_n) - 59\varphi(t_{n-1}, u_{n-1}) + 37\varphi(t_{n-2}, u_{n-2}) - 9\varphi(t_{n-3}, u_{n-3})\Big) \quad n = 3, 4, 5, \dots N-1 \end{cases}$$

[7]:
```
def AB4(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    uu.append(uu[1]+h*(3*phi(tt[1],uu[1])-phi(tt[0],uu[0]))/2)
    uu.
↪append(uu[2]+h*(23*phi(tt[2],uu[2])-16*phi(tt[1],uu[1])+5*phi(tt[0],uu[0]))/
↪12)
    for i in range(3,len(tt)-1):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        k3 = h * phi( tt[i-2], uu[i-2] )
        k4 = h * phi( tt[i-3], uu[i-3] )
        uu.append( uu[i] + (55*k1-59*k2+37*k3-9*k4) /24 )
    return uu
```

Schéma de Adam-Bashforth à 5 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ u_2 = u_1 + \frac{h}{2}\Big(3\varphi(t_1, u_1) - \varphi(t_0, u_0)\Big), \\ u_3 = u_2 + \frac{h}{12}\Big(23\varphi(t_2, u_2) - 16\varphi(t_1, u_1) + 5\varphi(t_0, u_0)\Big), \\ u_4 = u_3 + \frac{h}{24}\Big(55\varphi(t_3, u_3) - 59\varphi(t_2, u_2) + 37\varphi(t_1, u_1) - 9\varphi(t_0, u_0)\Big), \\ u_{n+1} = u_n + \frac{h}{720}\Big(1901\varphi(t_n, u_n) - 2774\varphi(t_{n-1}, u_{n-1}) + 2616\varphi(t_{n-2}, u_{n-2}) - 1274\varphi(t_{n-3}, u_{n-3}) + 251\varphi(t_{n-4}, u_{n-} \end{cases}$$

[8]:
```
def AB5(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    uu.append(uu[1]+h*(3*phi(tt[1],uu[1])-phi(tt[0],uu[0]))/2)
    uu.
 ↪append(uu[2]+h*(23*phi(tt[2],uu[2])-16*phi(tt[1],uu[1])+5*phi(tt[0],uu[0]))/
 ↪12)
    uu.
 ↪append(uu[3]+h*(55*phi(tt[3],uu[3])-59*phi(tt[2],uu[2])+37*phi(tt[1],uu[1])-9*phi(tt[0],uu[
 ↪24)
    for i in range(4,len(tt)-1):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        k3 = h * phi( tt[i-2], uu[i-2] )
        k4 = h * phi( tt[i-3], uu[i-3] )
        k5 = h * phi( tt[i-4], uu[i-4] )
        uu.append( uu[i] + (1901*k1-2774*k2+2616*k3-1274*k4+251*k5) /720 )
    return uu
```

Schéma de Nylström à 2 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ u_{n+1} = u_{n-1} + 2h\varphi(t_n, u_n) \quad n = 1, 2, 3, 4, 5, \dots N-1 \end{cases}$$

[9]:
```
def N2(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    for i in range(1,len(tt)-1):
        k1 = h * phi( tt[i], uu[i] )
        uu.append( uu[i-1] + 2*k1 )
    return uu
```

Schéma de Nylström à 3 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ u_2 = u_0 + 2h\varphi(t_1, u_1), \\ u_{n+1} = u_{n-1} + \frac{h}{3}\Big(7\varphi(t_n, u_n) - 2\varphi(t_{n-1}, u_{n-1}) + \varphi(t_{n-2}, u_{n-2})\Big) \quad n = 2, 3, 4, 5, \dots N-1 \end{cases}$$

[10]:
```
def N3(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    uu.append(uu[0]+2*h*phi(tt[1],uu[1]))
    for i in range(2,len(tt)-1):
        k1 = phi( tt[i], uu[i] )
        k2 = phi( tt[i-1], uu[i-1] )
        k3 = phi( tt[i-2], uu[i-2] )
        uu.append( uu[i-1] + (7*k1-2*k2+k3)*h/3 )
    return uu
```

Schéma de Nylström à 4 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ u_2 = u_0 + 2h\varphi(t_1, u_1), \\ u_3 = u_1 + \frac{h}{3}\Big(7\varphi(t_2, u_2) - 2\varphi(t_1, u_1) + \varphi(t_0, u_0)\Big), \\ u_{n+1} = u_{n-1} + \frac{h}{3}\Big(8\varphi(t_n, u_n) - 5\varphi(t_{n-1}, u_{n-1}) + 4\varphi(t_{n-2}, u_{n-2}) - \varphi(t_{n-3}, u_{n-3})\Big) \quad n = 3, 4, 5, \dots N-1 \end{cases}$$

[11]:
```
def N4(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    uu.append(uu[0]+2*h*phi(tt[1],uu[1]))
    uu.
↪append(uu[1]+(7*h*phi(tt[2],uu[2])-2*h*phi(tt[1],uu[1])+h*phi(tt[0],uu[0]))/
↪3)
    for i in range(3,len(tt)-1):
        k1 = phi( tt[i], uu[i] )
        k2 = phi( tt[i-1], uu[i-1] )
        k3 = phi( tt[i-2], uu[i-2] )
        k4 = phi( tt[i-3], uu[i-3] )
        uu.append( uu[i-1] + (8*k1-5*k2+4*k3-k4)*h/3 )
    return uu
```

Schéma de Runke-Kutta RK4

$$\begin{cases} u_0 = y(t_0) = y_0, \\ \tilde{u}_{n+1/2} = u_n + \frac{h}{2}\varphi(t_n, u_n), \\ \check{u}_{n+1/2} = u_n + \frac{h}{2}\varphi(t_n + \frac{h}{2}, \tilde{u}_{n+1/2}), \\ \hat{u}_{n+1} = u_n + h\varphi(t_{n+1}, \check{u}_{n+1/2}), \\ u_{n+1} = u_n + \frac{h}{6}\left(\varphi(t_n, u_n) + 2\varphi(t_n + \frac{h}{2}, \tilde{u}_{n+1/2}) + 2\varphi(t_n + \frac{h}{2}, \check{u}_{n+1/2}) + \varphi(t_{n+1}, \hat{u}_{n+1})\right) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

[38]:
```
def RK4(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        k1 = phi( tt[i], uu[i] )
        k2 = phi( tt[i]+h/2 , uu[i]+k1*h/2 )
        k3 = phi( tt[i]+h/2 , uu[i]+h*k2/2 )
        k4 = phi( tt[i+1]   , uu[i]+h*k3 )
        uu.append( uu[i] + (k1+2*k2+2*k3+k4)*h/6 )
    return uu
```

1.1.2 Schémas implicites

Schéma d'Euler régressif

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + h\varphi(t_{n+1}, u_{n+1}) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

avec u_{n+1} zéro de la fonction

$$x \mapsto -x + u_n + h\varphi(t_{n+1}, x)$$

[13]:
```
def euler_regressif(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        temp = fsolve(lambda x: -x+uu[i]+h*phi(tt[i+1],x), uu[i])
        uu.append(temp)
    return uu
```

Schéma de Crank-Nicolson

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + \frac{h}{2}\Big(\varphi(t_n, u_n) + \varphi(t_{n+1}, u_{n+1})\Big) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

avec u_{n+1}zéro de la fonction

$$x \mapsto -x + u_n + \frac{h}{2}(\varphi(t_n, u_n) + \varphi(t_{n+1}, x))$$

[14]:
```
def CN(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        temp = fsolve(lambda x: -x+uu[i]+0.5*h*(␣
↪phi(tt[i+1],x)+phi(tt[i],uu[i]) ), uu[i])
```

```
        uu.append(temp)
    return uu
```

Schéma de AM-2

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + \frac{h}{2}\Big(\varphi(t_1,u_1) + \varphi(t_0,u_0)\Big), \\ u_{n+1} = u_n + h\Big(5\varphi(t_{n+1},u_{n+1}) + 8\varphi(t_n,u_n) - \varphi(t_{n-1},u_{n-1})\Big) \quad n = 1,2,\dots N-1 \end{cases}$$

[15]:
```
def AM2(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    temp = fsolve(lambda x: -x+uu[0]+0.5*h*( phi(tt[1],x)+phi(tt[0],uu[0]) ),␣
↪uu[0])
    uu.append(temp)
    for i in range(1,len(tt)-1):
        temp = fsolve(lambda x: -x+uu[i]+h*(␣
↪5*phi(tt[i+1],x)+8*phi(tt[i],uu[i])-phi(tt[i-1],uu[i-1]) )/12, uu[i])
        uu.append(temp)
    return uu
```

Schéma de AM-3

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + \frac{h}{2}\Big(\varphi(t_1,u_1) + \varphi(t_0,u_0)\Big), \\ u_2 = u_n + h\Big(5\varphi(t_2,u_2) + 8\varphi(t_1,u_1) - \varphi(t_0,u_0)\Big), \\ u_{n+1} = u_n + h\Big(9\varphi(t_{n+1},u_{n+1}) + 19\varphi(t_n,u_n) - 5\varphi(t_{n-1},u_{n-1}) + \varphi(t_{n-2},u_{n-2})\Big) \quad n = 2,3,\dots N-1 \end{cases}$$

[16]:
```
def AM3(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    temp = fsolve(lambda x: -x+uu[0]+0.5*h*( phi(tt[1],x)+phi(tt[0],uu[0]) ),␣
↪uu[0])
    uu.append(temp)
    temp = fsolve(lambda x: -x+uu[1]+h*(␣
↪5*phi(tt[2],x)+8*phi(tt[1],uu[1])-phi(tt[0],uu[0]) )/12, uu[1])
    uu.append(temp)
    for i in range(2,len(tt)-1):
        temp = fsolve(lambda x: -x+uu[i]+h*(␣
↪9*phi(tt[i+1],x)+19*phi(tt[i],uu[i])-5*phi(tt[i-1],uu[i-1])+phi(tt[i-2],uu[i-2])␣
↪)/24, uu[i])
        uu.append(temp)
```

```
    return uu
```

Schéma de AM-4

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + \frac{h}{2}\Big(\varphi(t_1,u_1) + \varphi(t_0,u_0)\Big), \\ u_2 = u_1 + h\Big(5\varphi(t_2,u_2) + 8\varphi(t_1,u_1) - \varphi(t_0,u_0)\Big), \\ u_3 = u_2 + h\Big(9\varphi(t_3,u_3) + 19\varphi(t_2,u_2) - 5\varphi(t_1,u_1) + \varphi(t_0,u_0)\Big), \\ u_{n+1} = u_n + h\Big(251\varphi(t_{n+1},u_{n+1}) + 646\varphi(t_n,u_n) - 264\varphi(t_{n-1},u_{n-1}) + 106\varphi(t_{n-2},u_{n-2}) - 19\varphi(t_{n-3},u_{n-3})\Big) \quad n = \end{cases}$$

[17]:
```
def AM4(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    temp = fsolve(lambda x: -x+uu[0]+0.5*h*( phi(tt[1],x)+phi(tt[0],uu[0]) ),␣
↪uu[0])
    uu.append(temp)
    temp = fsolve(lambda x: -x+uu[1]+h*(␣
↪5*phi(tt[2],x)+8*phi(tt[1],uu[1])-phi(tt[0],uu[0]) )/12, uu[1])
    uu.append(temp)
    temp = fsolve(lambda x: -x+uu[2]+h*(␣
↪9*phi(tt[3],x)+19*phi(tt[2],uu[2])-5*phi(tt[1],uu[1])+phi(tt[0],uu[0]) )/24,␣
↪uu[1])
    uu.append(temp)
    for i in range(3,len(tt)-1):
        temp = fsolve(lambda x: -x+uu[i]+h*(␣
↪251*phi(tt[i+1],x)+646*phi(tt[i],uu[i])-264*phi(tt[i-1],uu[i-1])+106*phi(tt[i-2],uu[i-2])-1
↪)/720, uu[i])
        uu.append(temp)
    return uu
```

Schéma BDF2

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_1,u_1), \\ u_{n+1} = \frac{4}{3}u_n - \frac{1}{3}u_{n-1} + \frac{2}{3}\varphi(t_{n+1},u_{n+1}) \quad n = 1,2,\dots N-1 \end{cases}$$

[18]:
```
def BDF2(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    temp = fsolve(lambda x: -x+uu[0]+h*phi(tt[1],x), uu[0])
    uu.append(temp)
    for i in range(1,len(tt)-1):
```

```
        temp = fsolve(lambda x: -x+4/3*uu[i]-1/3*uu[i-1] + 2/3*h*phi(tt[i+1],x)␣
↪, uu[i])
        uu.append(temp)
    return uu
```

Schéma BDF3

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_1, u_1), \\ u_2 = \frac{4}{3}u_1 - \frac{1}{3}u_0 + \frac{2}{3}\varphi(t_2, u_2), \\ u_{n+1} = \frac{4}{3}u_n - \frac{1}{3}u_{n-1} + \frac{2}{3}\varphi(t_{n+1}, u_{n+1}) \quad n = 2, 3, \dots N-1 \end{cases}$$

[19]:
```
def BDF3(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    temp = fsolve(lambda x: -x+uu[0]+h*phi(tt[1],x), uu[0])
    uu.append(temp)
    temp = fsolve(lambda x: -x+4/3*uu[1]-1/3*uu[0] + 2/3*h*phi(tt[2],x), uu[1])
    uu.append(temp)
    for i in range(2,len(tt)-1):
        temp = fsolve(lambda x: -x+18/11*uu[i]-9/11*uu[i-1] + 2/11*uu[i-2]+6/
↪11*h*phi(tt[i+1],x) , uu[i])
        uu.append(temp)
    return uu
```

1.1.3 Schémas predicteur-correcteur

Schéma d'Euler modifié

$$\begin{cases} u_0 = y_0, \\ \tilde{u} = u_n + \frac{h}{2}\varphi(t_n, u_n), \\ u_{n+1} = u_n + h\varphi\left(t_n + \frac{h}{2}, \tilde{u}\right) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

[20]:
```
def euler_modifie(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        k1 = phi( tt[i], uu[i] )
        uu.append( uu[i]+h*phi(tt[i]+h/2,uu[i]+k1*h/2) )
    return uu
```

Schéma de Heun

$$\begin{cases} u_0 = y_0, \\ \tilde{u} = u_n + h\varphi(t_n, u_n) \\ u_{n+1} = u_n + \frac{h}{2}\Big(\varphi(t_n, u_n) + \varphi(t_{n+1}, \tilde{u})\Big) \end{cases} \quad n = 0, 1, 2, \dots N-1$$

[21]:
```
def heun(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    for i in range(len(tt)-1):
        k1 = phi( tt[i], uu[i] )
        k2 = phi( tt[i+1], uu[i] + h*k1  )
        uu.append( uu[i] + (k1+k2)*h/2 )
    return uu
```

Schéma AM-2 AB-1

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ \tilde{u} = u_n + h\varphi(t_n, u_n), \\ u_{n+1} = u_n + \frac{h}{12}\Big(5\varphi(t_{n+1}, \tilde{u}) + 8\varphi(t_n, u_n) - \varphi(t_{n-1}, u_{n-1})\Big) \end{cases} \quad n = 1, 2, \dots N-1$$

[22]:
```
def AM2AB1(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    for i in range(1,len(tt)-1):
        pred = uu[i] + h*phi(tt[i],uu[i])
        uu.
↪append(uu[i]+h*(5*phi(tt[i+1],pred)+8*phi(tt[i],uu[i])-phi(tt[i-1],uu[i-1]))/
↪12)
    return uu
```

Schéma AM-3 AB-2

$$\begin{cases} u_0 = y_0, \\ u_1 = u_0 + h\varphi(t_0, u_0), \\ u_2 = u_0 + \frac{h}{2}(3\varphi(t_1, u_1) - \varphi(t_0, u_0)), \\ \tilde{u} = u_n + \frac{h}{2}(3\varphi(t_n, u_n) - \varphi(t_{n-1}, u_{n-1})), \\ u_{n+1} = u_n + \frac{h}{24}\Big(9\varphi(t_{n+1}, \tilde{u}) + 19\varphi(t_n, u_n) - 5\varphi(t_{n-1}, u_{n-1}) + \varphi(t_{n-2}, u_{n-2})\Big) \end{cases} \quad n = 2, 3, \dots N-1$$

[23]:
```
def AM3AB2(phi,tt):
    h = tt[1]-tt[0]
    uu = [y0]
    uu.append(uu[0]+h*phi(tt[0],uu[0]))
    uu.append(uu[0]+0.5*h*(3*phi(tt[1],uu[1])-phi(tt[0],uu[0])))
    for i in range(2,len(tt)-1):
        k1 = phi( tt[i], uu[i] )
        k2 = phi( tt[i-1], uu[i-1] )
        pred = uu[i] + (3*k1-k2)*h/2
        uu.
↪append(uu[i]+h*(5*phi(tt[i+1],pred)+8*phi(tt[i],uu[i])-phi(tt[i-1],uu[i-1]))/
↪12)
    return uu
```

1.2 Comparaison sur un exemple

Considérons le problème de Cauchy >trouver la fonction $y\colon I \subset \mathbb{R} \to \mathbb{R}$ définie sur l'intervalle $I = [0, 1]$ telle que

$$\begin{cases} y'(t) = \sin(t) - y(t), & \forall t \in I = [0, 1], \\ y(0) = 0 \end{cases}$$

1. Calculer la solution exacte en utilisant le module `sympy`.
2. Calculer la solution approchée obtenue avec la méthode d'Euler explicite avec $h = 1/N$ et $N = 8$ (pour bien visualiser les erreurs);
3. même exercice pour les méthodes AB_2, AB_3, AB_4, AB_5, N_2, N_3, N_4, RK_4;
4. même exercice pour les méthodes d'Euler implicite, de Crank-Nicolson, AM_1, AM_2, AM_3, AM_4, BDF_2, BDF_3
5. même exercice pour les méthodes predictor-corrector Euler modifié, Heun, AM_2-AB_1, AM_3-AB_2.
6. Pour chaque méthode, afficher solution exacte vs solution approchée ainsi que le maximum de la valeur absolue de l'erreur.

Correction 1
Calculons la solutions exacte en utilisant le module `sympy`:

[39]:
```
import sympy as sym
sym.init_printing()

t  = sym.Symbol('t')
y  = sym.Function('y')
edo= sym.Eq( sym.diff(y(t),t) , sym.sin(t)-y(t) )
display(edo)
solgen = sym.dsolve(edo,y(t))
display(solgen)

t0=0
y0=0
```

```
consts = sym.solve( sym.Eq( y0, solgen.rhs.subs(t,t0)) , dict=True)[0]
display(consts)
solpar=solgen.subs(consts).simplify()
display(solpar)
```

$$\frac{d}{dt}y(t) = -y(t) + \sin(t)$$

$$y(t) = \left(C_1 + \frac{e^t}{2}\sin(t) - \frac{e^t}{2}\cos(t)\right)e^{-t}$$

$$\left\{C_1 : \frac{1}{2}\right\}$$

$$y(t) = -\frac{\sqrt{2}}{2}\cos\left(t + \frac{\pi}{4}\right) + \frac{e^{-t}}{2}$$

On définit la solution exacte à utiliser pour estimer les erreurs:

[40]:
```
sol_exacte = sym.lambdify(t,solpar.rhs,'numpy')
```

Correction de 2 à 5

On initialise le problème de Cauchy

[41]:
```
t0     = 0
tfinal = 1
y0 = 0
```

On définit l'équation différentielle : `phi` est une fonction python qui contient la fonction mathématique $\varphi(t, y) = sin(t) + y$ dépendant des variables t et y.

[42]:
```
phi = lambda t,y : sin(t) - y
```

On introduit la discrétisation: les $N + 1$ nœuds d'intégration $[t_0, t_1, \ldots, t_N]$ sont contenus dans le vecteur `tt`.
On a $N + 1$ points espacé de $h = \frac{t_N - t_0}{N}$.

[43]:
```
N = 8
tt = linspace(t0,tfinal,N+1)
```

On calcule les solutions exacte et approchées:

[44]:
```
yy = sol_exacte(tt)

uu_ep   = euler_progressif(phi,tt)
uu_AB2  = AB2(phi,tt)
uu_AB3  = AB3(phi,tt)
uu_AB4  = AB4(phi,tt)
```

```
uu_AB5  = AB5(phi,tt)
uu_N2   = N2(phi,tt)
uu_N3   = N3(phi,tt)
uu_N4   = N4(phi,tt)
uu_RK4  = RK4(phi,tt)

uu_er   = euler_regressif(phi,tt)
uu_CN   = CN(phi,tt)
uu_AM2  = AM2(phi,tt)
uu_AM3  = AM3(phi,tt)
uu_AM4  = AM4(phi,tt)
uu_BDF2 = BDF2(phi,tt)
uu_BDF3 = BDF3(phi,tt)

uu_em   = euler_modifie(phi,tt)
uu_heun = heun(phi,tt)
uu_AM2AB1 = AM2AB1(phi,tt)
uu_AM3AB2 = AM3AB2(phi,tt)
```

On compare les graphes des solutions exacte et approchées et on affiche le maximum de l'erreur:

[45]:
```
figure(1,figsize=(20,16))

subplot(4,5,1)
plot(tt,yy,'b-',tt,uu_ep,'r-D')
title('AB1=EE - max(|err|)=%1.3f'%(max([abs(uu_ep[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,2)
plot(tt,yy,'b-',tt,uu_AB2,'r-D')
title('AB2 - max(|err|)=%1.3f'%(max([abs(uu_AB2[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,3)
plot(tt,yy,'b-',tt,uu_AB3,'r-D')
title('AB3 - max(|err|)=%1.3f'%(max([abs(uu_AB3[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,4)
plot(tt,yy,'b-',tt,uu_AB4,'r-D')
title('AB4 - max(|err|)=%1.3f'%(max([abs(uu_AB4[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,5)
plot(tt,yy,'b-',tt,uu_AB5,'r-D')
title('AB5 - max(|err|)=%1.3f'%(max([abs(uu_AB5[i]-yy[i]) for i in␣
 ↪range(N+1)])))
```

```
subplot(4,5,6)
plot(tt,yy,'b-',tt,uu_N2,'r-D')
title('N2 - max(|err|)=%1.3f'%(max([abs(uu_N2[i]-yy[i]) for i in range(N+1)])))

subplot(4,5,7)
plot(tt,yy,'b-',tt,uu_N3,'r-D')
title('N3 - max(|err|)=%1.3f'%(max([abs(uu_N3[i]-yy[i]) for i in range(N+1)])))

subplot(4,5,8)
plot(tt,yy,'b-',tt,uu_N4,'r-D')
title('N4 - max(|err|)=%1.3f'%(max([abs(uu_N4[i]-yy[i]) for i in range(N+1)])))

subplot(4,5,9)
plot(tt,yy,'b-',tt,uu_RK4,'r-D')
title('RK4 - max(|err|)=%1.3f'%(max([abs(uu_RK4[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,10)
plot(tt,yy,'b-',tt,uu_er,'r-D')
title('AM0=EI - max(|err|)=%1.3f'%(max([abs(uu_er[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,11)
plot(tt,yy,'b-',tt,uu_CN,'r-D')
title('AM1=CN - max(|err|)=%1.3f'%(max([abs(uu_CN[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,12)
plot(tt,yy,'b-',tt,uu_AM2,'r-D')
title('AM2 - max(|err|)=%1.3f'%(max([abs(uu_AM2[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,13)
plot(tt,yy,'b-',tt,uu_AM3,'r-D')
title('AM3 - max(|err|)=%1.3f'%(max([abs(uu_AM3[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,14)
plot(tt,yy,'b-',tt,uu_AM4,'r-D')
title('AM4 - max(|err|)=%1.3f'%(max([abs(uu_AM4[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,15)
plot(tt,yy,'b-',tt,uu_BDF2,'r-D')
```

```
title('BDF2 - max(|err|)=%1.3f'%(max([abs(uu_BDF2[i]-yy[i]) for i in␣
 ↪range(N+1)])))

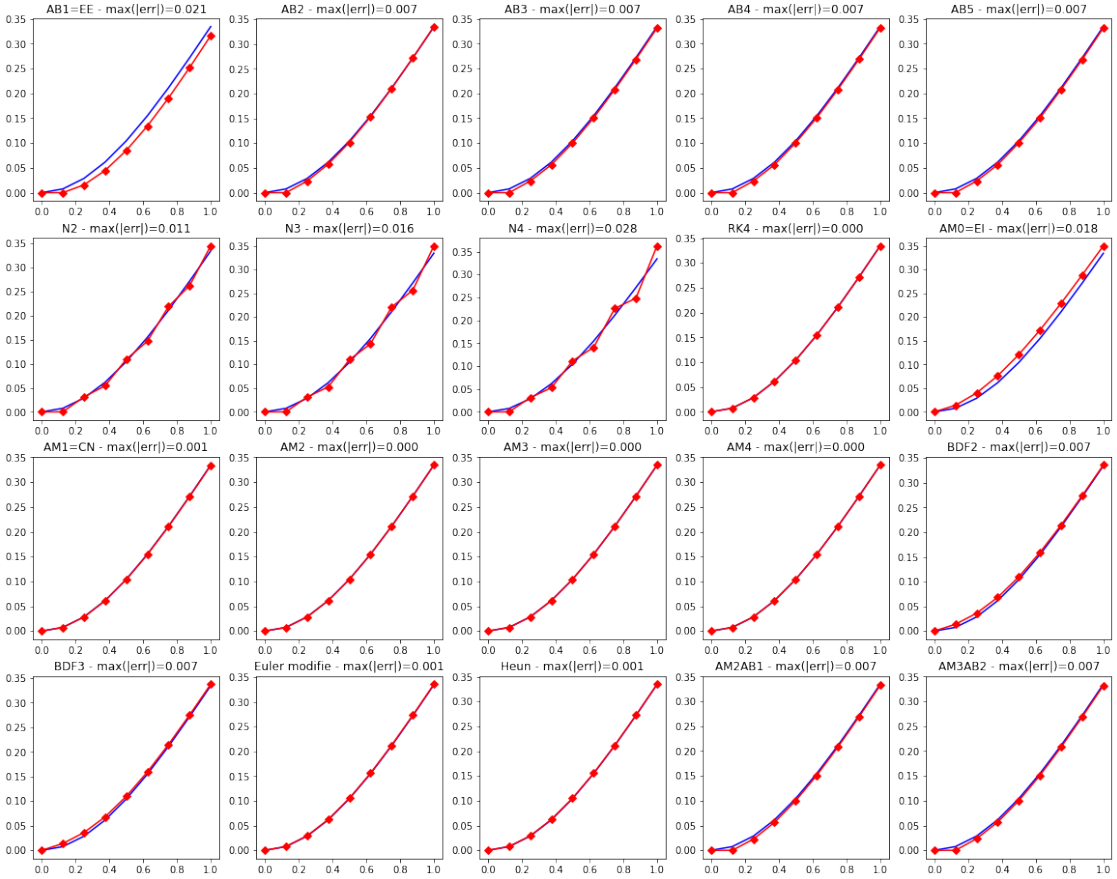
subplot(4,5,16)
plot(tt,yy,'b-',tt,uu_BDF3,'r-D')
title('BDF3 - max(|err|)=%1.3f'%(max([abs(uu_BDF3[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,17)
plot(tt,yy,'b-',tt,uu_em,'r-D')
title('Euler modifie - max(|err|)=%1.3f'%(max([abs(uu_em[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,18)
plot(tt,yy,'b-',tt,uu_heun,'r-D')
title('Heun - max(|err|)=%1.3f'%(max([abs(uu_heun[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,19)
plot(tt,yy,'b-',tt,uu_AM2AB1,'r-D')
title('AM2AB1 - max(|err|)=%1.3f'%(max([abs(uu_AM2AB1[i]-yy[i]) for i in␣
 ↪range(N+1)])))

subplot(4,5,20)
plot(tt,yy,'b-',tt,uu_AM3AB2,'r-D')
title('AM3AB2 - max(|err|)=%1.3f'%(max([abs(uu_AM3AB2[i]-yy[i]) for i in␣
 ↪range(N+1)])));
```


AB1=EE - max(|err|)=0.021
AB2 - max(|err|)=0.007
AB3 - max(|err|)=0.007
AB4 - max(|err|)=0.007
AB5 - max(|err|)=0.007
N2 - max(|err|)=0.011
N3 - max(|err|)=0.016
N4 - max(|err|)=0.028
RK4 - max(|err|)=0.000
AM0=EI - max(|err|)=0.018
AM1=CN - max(|err|)=0.001
AM2 - max(|err|)=0.000
AM3 - max(|err|)=0.000
AM4 - max(|err|)=0.000
BDF2 - max(|err|)=0.007
BDF3 - max(|err|)=0.007
Euler modifie - max(|err|)=0.001
Heun - max(|err|)=0.001
AM2AB1 - max(|err|)=0.007
AM3AB2 - max(|err|)=0.007

[ ]:

TP4-Multisteps-conv

March 16, 2020

[1]:
```
from IPython.display import display, Latex
from IPython.core.display import HTML
css_file = './custom.css'
HTML(open(css_file, "r").read())
```

[1]:
```
<IPython.core.display.HTML object>
```

[2]:
```
import sys #only needed to determine Python version number
print('Python version ' + sys.version)
```

```
Python version 3.6.9 (default, Nov  7 2019, 10:44:02)
[GCC 8.3.0]
```

1 M62 TP 4 - Étude de la convergence

[3]:
```
%reset -f
%matplotlib inline
%autosave 300

from matplotlib.pylab import *
from scipy.optimize import fsolve
```

```
Autosaving every 300 seconds
```

2 Étude de la convergence

Considérons le problème de Cauchy

trouver la fonction $y\colon I \subset \mathbb{R} \to \mathbb{R}$ définie sur l'intervalle $I = [0, 1]$ telle que

$$\begin{cases} y'(t) = y(t), & \forall t \in I = [0, 1], \\ y(0) = 1 \end{cases}$$

dont la solution est $y(t) = e^t$.

On se propose d'estimer l'ordre de convergence d'une méthode numérique. + Pour chaque schéma, on calcule la solution approchée avec différentes valeurs de $h_k = 1/N_k$. On sauvegarde les valeurs de h_k dans le vecteur `H`. + Pour chaque valeur de h_k, on calcule le maximum de la valeur absolue de l'erreur et on sauvegarde toutes ces erreurs dans le vecteur `err_schema` de sort que `err_schema[k]` contient $e_k = \max_{i=0,\dots,N_k} |y(t_i) - u_i|$. + Pour afficher l'ordre de convergence on utilise une échelle logarithmique, i.e. on représente $\ln(h)$ sur l'axe des abscisses et $\ln(\text{err})$ sur l'axe des ordonnées.
En effet, si $\text{err} = Ch^p$ alors $\ln(\text{err}) = \ln(C) + p\ln(h)$.
En échelle logarithmique, p représente donc la pente de la ligne droite $\ln(\text{err})$.

Remarque: puisque la fonction $\varphi(t, y) = y$ est linéaire, toute méthode implicite peut être rendue explicite par un calcul élémentaire en explicitant directement pour chaque schéma l'expression de u_{n+1}. Cependant, nous pouvons utiliser le le module `SciPy` sans modifier l'implémentation des schémas (mais on payera l'ordre de convergence de `fsolve`).

Estimer l'ordre de convergence des méthodes: 1. EE, AB_2, AB_3, AB_4, AB_5, N_2, N_3, N_4 2. EI, CN, AM_1, AM_2, AM_3, AM_4, BDF_2, BDF_3 3. EM, Heun, RK4-1, RK4-2 4. AM_4-AB_1, AM_4-AB_2, AM_4-AB_3

Attention: les schémas multistep ont besoin d'initialiser plusieurs pas de la suite définie pas récurrence pour pouvoir démarrer. Dans cette étude, au lieu d'utiliser un schéma d'ordre inférieur pour initialiser la suite, on utilisera la solution exacte (en effet, l'utilisation d'un schéma d'ordre inférieur dégrade l'ordre de précision).

On écrit les schémas numériques : + les nœuds d'intégration $[t_0, t_1, \dots, t_N]$ sont contenus dans le vecteur `tt` (qui change en fonction de `h`) + les valeurs $[u_0, u_1, \dots, u_N]$ pour chaque méthode sont contenues dans le vecteur `uu`.

2.1 Schémas explicites

2.1.1 Schéma de Adam-Bashforth à 1 pas = schéma d'Euler progressif

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + h\varphi(t_n, u_n) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

[4]:
```
def euler_progressif(phi,tt):
    uu = [y0]
    for i in range(N):
        uu.append(uu[i]+h*phi(tt[i],uu[i]))
    return uu
```

2.1.2 Schéma de Adam-Bashforth à 2 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_{n+1} = u_n + \frac{h}{2}\Big(3\varphi(t_n, u_n) - \varphi(t_{n-1}, u_{n-1})\Big) \quad n = 1, 2, 3, 4, 5, \dots N-1 \end{cases}$$

[5]:
```
def AB2(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    for i in range(1,N):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        uu.append( uu[i] + (3.*k1-k2) /2.0 )
    return uu
```

2.1.3 Schéma de Adam-Bashforth à 3 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_{n+1} = u_n + \frac{h}{12}\Big(23\varphi(t_n,u_n) - 16\varphi(t_{n-1},u_{n-1}) + 5\varphi(t_{n-2},u_{n-2})\Big) \quad n = 2,3,4,5,\dots N-1 \end{cases}$$

[6]:
```
def AB3(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    uu.append(sol_exacte(tt[2]))
    for i in range(2,N):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        k3 = h * phi( tt[i-2], uu[i-2] )
        uu.append( uu[i] + (23.*k1-16.*k2+5.*k3) /12.0 )
    return uu
```

2.1.4 Schéma de Adam-Bashforth à 4 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_3 = y_3, \\ u_{n+1} = u_n + \frac{h}{24}\Big(55\varphi(t_n,u_n) - 59\varphi(t_{n-1},u_{n-1}) + 37\varphi(t_{n-2},u_{n-2}) - 9\varphi(t_{n-3},u_{n-3})\Big) \quad n = 3,4,5,\dots N-1 \end{cases}$$

[7]:
```
def AB4(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    uu.append(sol_exacte(tt[2]))
    uu.append(sol_exacte(tt[3]))
    for i in range(3,N):
        k1 = h * phi( tt[i], uu[i] )
```

```
        k2 = h * phi( tt[i-1], uu[i-1] )
        k3 = h * phi( tt[i-2], uu[i-2] )
        k4 = h * phi( tt[i-3], uu[i-3] )
        uu.append( uu[i] + (55.*k1-59.*k2+37.*k3-9.*k4) /24.0 )
    return uu
```

2.1.5 Schéma de Adam-Bashforth à 5 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_3 = y_3, \\ u_4 = y_4, \\ u_{n+1} = u_n + \frac{h}{720}\Big(1901\varphi(t_n,u_n) - 2774\varphi(t_{n-1},u_{n-1}) + 2616\varphi(t_{n-2},u_{n-2}) - 1274\varphi(t_{n-3},u_{n-3}) + 251\varphi(t_{n-4},u_{n-4}\end{cases}$$

[8]:
```
def AB5(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    uu.append(sol_exacte(tt[2]))
    uu.append(sol_exacte(tt[3]))
    uu.append(sol_exacte(tt[4]))
    for i in range(4,N):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        k3 = h * phi( tt[i-2], uu[i-2] )
        k4 = h * phi( tt[i-3], uu[i-3] )
        k5 = h * phi( tt[i-4], uu[i-4] )
        uu.append( uu[i] + (1901.*k1-2774.*k2+2616.*k3-1274.*k4+251*k5) /720.0 )
    return uu
```

2.1.6 Schéma de Nylström à 2 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_{n+1} = u_{n-1} + 2h\varphi(t_n,u_n) \quad n = 1,2,3,4,5,\dots N-1 \end{cases}$$

[9]:
```
def N2(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    for i in range(1,N):
        k1 = h * phi( tt[i], uu[i] )
        uu.append( uu[i-1] + 2.0*k1 )
    return uu
```

2.1.7 Schéma de Nylström à 3 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_{n+1} = u_{n-1} + \frac{h}{3}\Big(7\varphi(t_n, u_n) - 2\varphi(t_{n-1}, u_{n-1}) + \varphi(t_{n-2}, u_{n-2})\Big) \quad n = 2, 3, 4, 5, \dots N-1 \end{cases}$$

[10]:
```
def N3(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    uu.append(sol_exacte(tt[2]))
    for i in range(2,N):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        k3 = h * phi( tt[i-2], uu[i-2] )
        uu.append( uu[i-1] + (7.*k1-2.*k2+k3)/3. )
    return uu
```

2.1.8 Schéma de Nylström à 4 pas

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_3 = y_3, \\ u_{n+1} = u_{n-1} + \frac{h}{3}\Big(8\varphi(t_n, u_n) - 5\varphi(t_{n-1}, u_{n-1}) + 4\varphi(t_{n-2}, u_{n-2}) - \varphi(t_{n-3}, u_{n-3})\Big) \quad n = 3, 4, 5, \dots N-1 \end{cases}$$

[11]:
```
def N4(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    uu.append(sol_exacte(tt[2]))
    uu.append(sol_exacte(tt[3]))
    for i in range(3,N):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i-1], uu[i-1] )
        k3 = h * phi( tt[i-2], uu[i-2] )
        k4 = h * phi( tt[i-3], uu[i-3] )
        uu.append( uu[i-1] + (8.*k1-5.*k2+4.*k3-k4)/3. )
    return uu
```

2.1.9 Schéma de Runge-Kutta RK4-1

$$\begin{cases} u_0 = y(t_0) = y_0, \\ \tilde{u}_{n+1/2} = u_n + \frac{h}{2}\varphi(t_n, u_n), \\ \check{u}_{n+1/2} = u_n + \frac{h}{2}\varphi(t_n + \frac{h}{2}, \tilde{u}_{n+1/2}), \\ \hat{u}_{n+1} = u_n + h\varphi(t_{n+1}, \check{u}_{n+1/2}), \\ u_{n+1} = u_n + \frac{h}{6}\left(\varphi(t_n, u_n) + 2\varphi(t_n + \frac{h}{2}, \tilde{u}_{n+1/2}) + 2\varphi(t_n + \frac{h}{2}, \check{u}_{n+1/2}) + \varphi(t_{n+1}, \hat{u}_{n+1})\right) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

[12]:
```
def RK4(phi,tt):
    uu = [y0]
    for i in range(N):
        k1 = phi( tt[i], uu[i] )
        k2 = phi( tt[i]+h/2 , uu[i]+h*k1/2 )
        k3 = phi( tt[i]+h/2 , uu[i]+h*k2/2 )
        k4 = phi( tt[i+1]    , uu[i]+h*k3 )
        uu.append( uu[i] + h*(k1+2*k2+2*k3+k4) /6 )
    return uu
```

2.2 Schémas implicites

2.2.1 Schéma d'Euler régressif

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + h\varphi(t_{n+1}, u_{n+1}) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

avec u_{n+1} zéro de la fonction

$$x \mapsto -x + u_n + h\varphi(t_{n+1}, x).$$

[13]:
```
def euler_regressif(phi,tt):
    uu = [y0]
    for i in range(N):
        temp = fsolve(lambda x: -x+uu[i]+h*phi(tt[i+1],x), uu[i])
        uu.append(temp)
    return uu
```

2.2.2 Schéma de Crank-Nicolson

$$\begin{cases} u_0 = y_0, \\ u_{n+1} = u_n + \frac{h}{2}\Big(\varphi(t_n, u_n) + \varphi(t_{n+1}, u_{n+1})\Big) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

avec u_{n+1} un zéro de la fonction $x \mapsto -x + u_n + \frac{h}{2}(\varphi(t_n, u_n) + \varphi(t_{n+1}, x))$.

[14]:
```
def CN(phi,tt):
    uu = [y0]
    for i in range(len(tt)-1):
```

```
        temp = fsolve(lambda x: -x+uu[i]+0.5*h*(␣
↪phi(tt[i+1],x)+phi(tt[i],uu[i]) ), uu[i])
        uu.append(temp)
    return uu
```

2.2.3 Schéma de AM-2

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_{n+1} = u_n + h\Big(5\varphi(t_{n+1}, u_{n+1}) + 8\varphi(t_n, u_n) - \varphi(t_{n-1}, u_{n-1})\Big) \quad n = 1, 2, \dots N-1 \end{cases}$$

[15]:
```
def AM2(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    for i in range(1,N):
        temp = fsolve(lambda x: -x+uu[i]+h*( 5.*phi(tt[i+1],x)+8.
↪*phi(tt[i],uu[i])-phi(tt[i-1],uu[i-1]) )/12., uu[i])
        uu.append(temp)
    return uu
```

2.2.4 Schéma de AM-3

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_{n+1} = u_n + h\Big(9\varphi(t_{n+1}, u_{n+1}) + 19\varphi(t_n, u_n) - 5\varphi(t_{n-1}, u_{n-1}) + \varphi(t_{n-2}, u_{n-2})\Big) \quad n = 2, 3, \dots N-1 \end{cases}$$

[16]:
```
def AM3(phi,tt):
        uu = [y0]
        uu.append(sol_exacte(tt[1]))
        uu.append(sol_exacte(tt[2]))
        for i in range(2,N):
                temp = fsolve(lambda x: -x+uu[i]+h*( 9.*phi(tt[i+1],x)+19.
↪*phi(tt[i],uu[i])-5.*phi(tt[i-1],uu[i-1])+phi(tt[i-2],uu[i-2]) )/24., uu[i])
                uu.append(temp)
        return uu
```

2.2.5 Schéma de AM-4

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_3 = y_3, \\ u_{n+1} = u_n + h\Big(251\varphi(t_{n+1}, u_{n+1}) + 646\varphi(t_n, u_n) - 264\varphi(t_{n-1}, u_{n-1}) + 106\varphi(t_{n-2}, u_{n-2}) - 19\varphi(t_{n-3}, u_{n-3})\Big) \quad n = \end{cases}$$

[17]:
```
def AM4(phi,tt):
        uu = [y0]
        uu.append(sol_exacte(tt[1]))
        uu.append(sol_exacte(tt[2]))
        uu.append(sol_exacte(tt[3]))
        for i in range(3,N):
                temp = fsolve(lambda x: -x+uu[i]+h*( 251.*phi(tt[i+1],x)+646.
↪*phi(tt[i],uu[i])-264.*phi(tt[i-1],uu[i-1])+106.*phi(tt[i-2],uu[i-2])-19.
↪*phi(tt[i-3],uu[i-3]) )/720., uu[i])
                uu.append(temp)
        return uu
```

2.2.6 Schéma de AM-5

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_3 = y_3, \\ u_4 = y_4, \\ u_{n+1} = u_n + h\Big(475\varphi(t_{n+1}, u_{+1}) + 1427\varphi(t_n, u_n) - 798\varphi(t_{n-1}, u_{n-1}) + 482\varphi(t_{n-2}, u_{n-2}) - 173\varphi(t_{n-3}, u_{n-3}) + 27 \end{cases}$$

[18]:
```
def AM5(phi,tt):
        uu = [y0]
        uu.append(sol_exacte(tt[1]))
        uu.append(sol_exacte(tt[2]))
        uu.append(sol_exacte(tt[3]))
        uu.append(sol_exacte(tt[4]))
        for i in range(4,N):
                temp = fsolve(lambda x: -x+uu[i]+h*( 475.*phi(tt[i+1],x)+1427.
↪*phi(tt[i],uu[i])-798.*phi(tt[i-1],uu[i-1])+482.*phi(tt[i-2],uu[i-2])-173.
↪*phi(tt[i-3],uu[i-3])+27.*phi(tt[i-4],uu[i-4]))/1440., uu[i])
                uu.append(temp)
        return uu
```

2.2.7 Schéma MS-2

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_{n+1} = u_{n-1} + \frac{h}{3}\Big(\varphi(t_{n+1}, u_{n+1}) + 4\varphi(t_n, u_n) + \varphi(t_{n-1}, u_{n-1})\Big) \quad n = 1, 2, \dots N-1 \end{cases}$$

[46]:
```
def MS2(phi,tt):
    uu = [y0]
    uu.append(sol_exacte(tt[1]))
    for i in range(1,N):
        temp = fsolve(lambda x:␣
↪-x+uu[i-1]+h*(phi(tt[i+1],x)+4*phi(tt[i],uu[i])+phi(tt[i-1],uu[i-1]) )/3,␣
↪uu[i])
        uu.append(temp)
    return uu
```

2.2.8 Schéma BDF2

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_{n+1} = \frac{4}{3}u_n - \frac{1}{3}u_{n-1} + \frac{2}{3}\varphi(t_{n+1}, u_{n+1}) \quad n = 1, 2, \dots N-1 \end{cases}$$

[19]:
```
def BDF2(phi,tt):
        uu = [y0]
        uu.append(sol_exacte(tt[1]))
        for i in range(1,N):
                temp = fsolve(lambda x: -x+4./3.*uu[i]-1./3.*uu[i-1] + 2./3.
↪*h*phi(tt[i+1],x) , uu[i])
                uu.append(temp)
        return uu
```

2.2.9 Schéma BDF3

$$\begin{cases} u_0 = y_0, \\ u_1 = y_1, \\ u_2 = y_2, \\ u_{n+1} = \frac{4}{3}u_n - \frac{1}{3}u_{n-1} + \frac{2}{3}\varphi(t_{n+1}, u_{n+1}) \quad n = 2, 3, \dots N-1 \end{cases}$$

[20]:
```
def BDF3(phi,tt):
        uu = [y0]
        uu.append(sol_exacte(tt[1]))
        uu.append(sol_exacte(tt[2]))
        for i in range(2,N):
```

```
                temp = fsolve(lambda x: -x+18./11.*uu[i]-9./11.*uu[i-1] + 2./11.
↪*uu[i-2]+6./11.*h*phi(tt[i+1],x) , uu[i])
                uu.append(temp)
        return uu
```

2.3 Schémas predicteur-correcteur

2.3.1 Schéma d'Euler modifié

$$\begin{cases} u_0 = y_0, \\ \tilde{u} = u_n + \frac{h}{2}\varphi(t_n, u_n), \\ u_{n+1} = u_n + h\varphi\left(t_n + \frac{h}{2}, \tilde{u}\right) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

[22]:
```
def euler_modifie(phi,tt):
    uu = [y0]
    for i in range(N):
        k1 = h * phi( tt[i], uu[i] )
        uu.append( uu[i]+h*phi(tt[i]+h/2.,uu[i]+k1/2.) )
    return uu
```

2.3.2 Schéma de Heun

$$\begin{cases} u_0 = y_0, \\ \tilde{u} = u_n + h\varphi(t_n, u_n) \\ u_{n+1} = u_n + \frac{h}{2}\Big(\varphi(t_n, u_n) + \varphi(t_{n+1}, \tilde{u})\Big) \quad n = 0, 1, 2, \dots N-1 \end{cases}$$

[23]:
```
def heun(phi,tt):
    uu = [y0]
    for i in range(N):
        k1 = h * phi( tt[i], uu[i] )
        k2 = h * phi( tt[i+1], uu[i] + k1  )
        uu.append( uu[i] + (k1+k2) /2.0 )
    return uu
```

2.3.3 Schéma AM-4 AB-2/3/4

[26]:
```
def AM4AB2(phi,tt):
        uu = [y0]
        uu.append(sol_exacte(tt[1]))
        uu.append(sol_exacte(tt[2]))
        uu.append(sol_exacte(tt[3]))
        for i in range(3,N):
```

```
                k1 = h * phi( tt[i], uu[i] )
                k2 = h * phi( tt[i-1], uu[i-1] )
                pred = uu[i] + (3.*k1-k2) /2.0
                uu.append(uu[i]+h*( 251.*phi(tt[i+1],pred)+646.
 ↪*phi(tt[i],uu[i])-264.*phi(tt[i-1],uu[i-1])+106.*phi(tt[i-2],uu[i-2])-19.
 ↪*phi(tt[i-3],uu[i-3]) )/720.)
        return uu

def AM4AB3(phi,tt):
        uu = [y0]
        uu.append(sol_exacte(tt[1]))
        uu.append(sol_exacte(tt[2]))
        uu.append(sol_exacte(tt[3]))
        for i in range(3,N):
                k1 = h * phi( tt[i], uu[i] )
                k2 = h * phi( tt[i-1], uu[i-1] )
                k3 = h * phi( tt[i-2], uu[i-2] )
                pred = uu[i] + (23.*k1-16.*k2+5.*k3) /12.0
                uu.append(uu[i]+h*( 251.*phi(tt[i+1],pred)+646.
 ↪*phi(tt[i],uu[i])-264.*phi(tt[i-1],uu[i-1])+106.*phi(tt[i-2],uu[i-2])-19.
 ↪*phi(tt[i-3],uu[i-3]) )/720.)
        return uu

def AM4AB4(phi,tt):
        uu = [y0]
        uu.append(sol_exacte(tt[1]))
        uu.append(sol_exacte(tt[2]))
        uu.append(sol_exacte(tt[3]))
        for i in range(3,N):
                k1 = h * phi( tt[i], uu[i] )
                k2 = h * phi( tt[i-1], uu[i-1] )
                k3 = h * phi( tt[i-2], uu[i-2] )
                k4 = h * phi( tt[i-3], uu[i-3] )
                pred = uu[i] + (55.*k1-59.*k2+37.*k3-9.*k4) /24.0
                uu.append(uu[i]+h*( 251.*phi(tt[i+1],pred)+646.
 ↪*phi(tt[i],uu[i])-264.*phi(tt[i-1],uu[i-1])+106.*phi(tt[i-2],uu[i-2])-19.
 ↪*phi(tt[i-3],uu[i-3]) )/720.)
        return uu
```

2.4 Convergence

On initialise le problème de Cauchy

[47]:
```
t0, y0, tfinal = 0, 1, 3
```

On définit la solution exacte:

[48]:
```
def sol_exacte(t):
        return exp(t)
        #return exp(-t)
        #return sqrt(2.*t+1.)
        #return sqrt(t**2+1.)
        #return 1./sqrt(1.-2.*t)
```

On définit l'équation différentielle : `phi` est une fonction python qui contient la fonction mathématique $\varphi(t, y)$ dépendant des variables t et y.

[49]:
```
def phi(t,y):
        return y
        #return -y
        #return 1./y
        #return t/y
        #return y**3
```

Pour chaque schéma, on calcule la solution approchée avec différentes valeurs de $h_k = 1/N_k$, à savoir $N_k = 2, 2^2, 2^3, \ldots 2^{10}$). On sauvegarde les valeurs de h_k dans le vecteur `H`.

Pour chaque valeur de h_k, on calcule le maximum de la valeur absolue de l'erreur et on sauvegarde toutes ces erreurs dans le vecteur `err_schema` de sort que `err_schema[k]` contient $e_k = \max_{i=0,\ldots,N_k} |y(t_i) - u_i|$ avec $N_k = 2^{k+1}$.

[50]:
```
H = []

err_ep   = []
err_AB2  = []
err_AB3  = []
err_AB4  = []
err_AB5  = []
err_N2   = []
err_N3   = []
err_N4   = []
err_RK4  = []

err_er   = []
err_CN   = []
err_AM2  = []
err_AM3  = []
err_AM4  = []
err_AM5  = []
err_BDF2 = []
err_BDF3 = []
err_MS2=[]

err_em   = []
err_heun = []
```

```
err_AM4AB2  = []
err_AM4AB3  = []
err_AM4AB4  = []

for k in range(8):
        N = 2**(k+3)
        h = (tfinal-t0)/N
        H.append(h)
        tt = [t0+i*h for i in range(N+1)]
        yy = [sol_exacte(t) for t in tt]
        # schemas explicites
        uu_ep   = euler_progressif(phi,tt)
        uu_AB2  = AB2(phi,tt)
        uu_AB3  = AB3(phi,tt)
        uu_AB4  = AB4(phi,tt)
        uu_AB5  = AB5(phi,tt)
        uu_N2   = N2(phi,tt)
        uu_N3   = N3(phi,tt)
        uu_N4   = N4(phi,tt)
        uu_RK4  = RK4(phi,tt)
        # schemas implicites
        uu_er   = euler_regressif(phi,tt)
        uu_CN   = CN(phi,tt)
        uu_AM2  = AM2(phi,tt)
        uu_AM3  = AM3(phi,tt)
        uu_AM4  = AM4(phi,tt)
        uu_AM5  = AM5(phi,tt)
        uu_BDF2 = BDF2(phi,tt)
        uu_BDF3 = BDF3(phi,tt)
        uu_MS2  = MS2(phi,tt)
        # schemas predictor-corrector
        uu_em   = euler_modifie(phi,tt)
        uu_heun = heun(phi,tt)
        uu_AM4AB2 = AM4AB2(phi,tt)
        uu_AM4AB3 = AM4AB3(phi,tt)
        uu_AM4AB4 = AM4AB4(phi,tt)
        # erreurs
        err_ep.append(max([abs(uu_ep[i]-yy[i]) for i in range(N+1)]))
        err_AB2.append(max([abs(uu_AB2[i]-yy[i]) for i in range(N+1)]))
        err_AB3.append(max([abs(uu_AB3[i]-yy[i]) for i in range(N+1)]))
        err_AB4.append(max([abs(uu_AB4[i]-yy[i]) for i in range(N+1)]))
        err_AB5.append(max([abs(uu_AB5[i]-yy[i]) for i in range(N+1)]))
        err_N2.append(max([abs(uu_N2[i]-yy[i]) for i in range(N+1)]))
        err_N3.append(max([abs(uu_N3[i]-yy[i]) for i in range(N+1)]))
        err_N4.append(max([abs(uu_N4[i]-yy[i]) for i in range(N+1)]))
        err_RK4.append(max([abs(uu_RK4[i]-yy[i]) for i in range(N+1)]))
```

```
        err_er.append(max([abs(uu_er[i]-yy[i]) for i in range(N+1)]))
        err_CN.append(max([abs(uu_CN[i]-yy[i]) for i in range(N+1)]))
        err_AM2.append(max([abs(uu_AM2[i]-yy[i]) for i in range(N+1)]))
        err_AM3.append(max([abs(uu_AM3[i]-yy[i]) for i in range(N+1)]))
        err_AM4.append(max([abs(uu_AM4[i]-yy[i]) for i in range(N+1)]))
        err_AM5.append(max([abs(uu_AM5[i]-yy[i]) for i in range(N+1)]))
        err_BDF2.append(max([abs(uu_BDF2[i]-yy[i]) for i in range(N+1)]))
        err_BDF3.append(max([abs(uu_BDF3[i]-yy[i]) for i in range(N+1)]))
        err_MS2.append(max([abs(uu_MS2[i]-yy[i]) for i in range(N+1)]))

        err_em.append(max([abs(uu_em[i]-yy[i]) for i in range(N+1)]))
        err_heun.append(max([abs(uu_heun[i]-yy[i]) for i in range(N+1)]))
        err_AM4AB2.append(max([abs(uu_AM4AB2[i]-yy[i]) for i in range(N+1)]))
        err_AM4AB3.append(max([abs(uu_AM4AB3[i]-yy[i]) for i in range(N+1)]))
        err_AM4AB4.append(max([abs(uu_AM4AB4[i]-yy[i]) for i in range(N+1)]))
```

Pour estimer l'ordre de convergence on estime la pente de la droite qui relie l'erreur au pas k à l'erreur au pas $k+1$ en echelle logarithmique en utilisant la fonction `polyfit` basée sur la régression linéaire.

[51]:
```
print ('EE\t %1.2f' %(polyfit(log(H),log(err_ep), 1)[0]))
print ('AB2\t %1.2f' %(polyfit(log(H),log(err_AB2), 1)[0]))
print ('AB3\t %1.2f' %(polyfit(log(H),log(err_AB3), 1)[0]))
print ('AB4\t %1.2f' %(polyfit(log(H),log(err_AB4), 1)[0]))
print ('AB5\t %1.2f' %(polyfit(log(H),log(err_AB5), 1)[0]))
print ('N2\t %1.2f' %(polyfit(log(H),log(err_N2), 1)[0]))
print ('N3\t %1.2f' %(polyfit(log(H),log(err_N3), 1)[0]))
print ('N4\t %1.2f' %(polyfit(log(H),log(err_N4), 1)[0]))
print ('RK4\t %1.2f' %(polyfit(log(H),log(err_RK4), 1)[0]))
print('\n')
print ('EI\t %1.2f' %(polyfit(log(H),log(err_er), 1)[0]))
print ('CN\t %1.2f' %(polyfit(log(H),log(err_CN), 1)[0]))
print ('AM2\t %1.2f' %(polyfit(log(H),log(err_AM2), 1)[0]))
print ('AM3\t %1.2f' %(polyfit(log(H),log(err_AM3), 1)[0]))
print ('AM4\t %1.2f' %(polyfit(log(H),log(err_AM4), 1)[0]))
print ('AM5\t %1.2f' %(polyfit(log(H),log(err_AM5), 1)[0]))
print ('BDF2\t %1.2f' %(polyfit(log(H),log(err_BDF2), 1)[0]))
print ('BDF3\t %1.2f' %(polyfit(log(H),log(err_BDF3), 1)[0]))
print ('MS2\t %1.2f' %(polyfit(log(H),log(err_MS2), 1)[0]))
print('\n')
print ('EM\t %1.2f' %(polyfit(log(H),log(err_em), 1)[0]))
print ('Heun\t %1.2f' %(polyfit(log(H),log(err_heun), 1)[0]))
print ('AM4AB2\t %1.2f' %(polyfit(log(H),log(err_AM4AB2), 1)[0]))
print ('AM4AB3\t %1.2f' %(polyfit(log(H),log(err_AM4AB3), 1)[0]))
print ('AM4AB4\t %1.2f' %(polyfit(log(H),log(err_AM4AB4), 1)[0]))
```

```
EE      0.92
AB2     1.92
```

```
AB3      2.87
AB4      3.82
AB5      4.75
N2       1.97
N3       2.92
N4       3.86
RK4      3.95


EI       1.12
CN       2.01
AM2      2.95
AM3      3.90
AM4      4.85
AM5      5.28
BDF2     1.95
BDF3     2.87
MS2      3.98


EM       1.95
Heun     1.95
AM4AB2   2.86
AM4AB3   3.82
AM4AB4   4.80
```

Pour afficher l'ordre de convergence on utilise une échelle logarithmique : on représente $\ln(h)$ sur l'axe des abscisses et $\ln(\text{err})$ sur l'axe des ordonnées. Le but de cette représentation est clair: si $\text{err} = Ch^p$ alors $\ln(\text{err}) = \ln(C) + p\ln(h)$. En échelle logarithmique, p représente donc la pente de la ligne droite $\ln(\text{err})$.

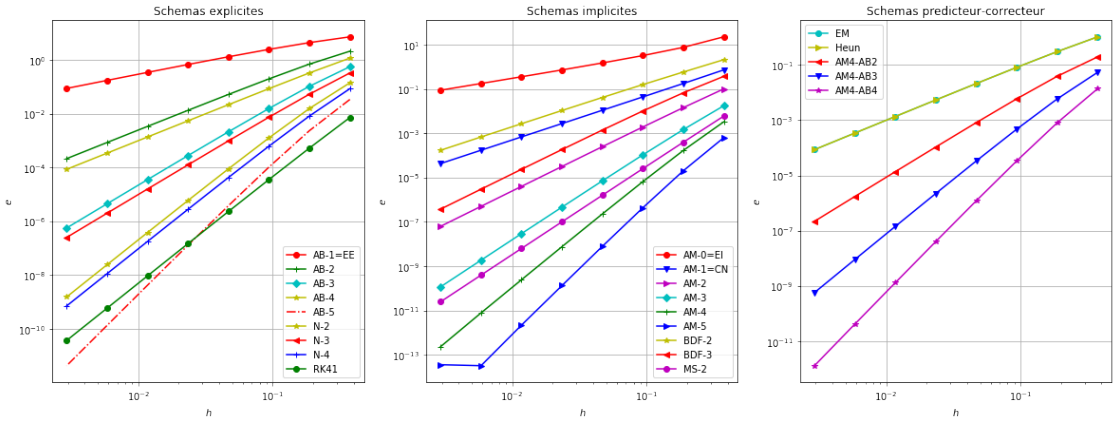
[53]:
```
figure(figsize=(20,7))

subplot(1,3,1)
loglog(H,err_ep, 'r-o',label='AB-1=EE')
loglog(H,err_AB2, 'g-+',label='AB-2')
loglog(H,err_AB3, 'c-D',label='AB-3')
loglog(H,err_AB4, 'y-*',label='AB-4')
loglog(H,err_AB5, 'r-.',label='AB-5')
loglog(H,err_N2, 'y-*',label='N-2')
loglog(H,err_N3, 'r-<',label='N-3')
loglog(H,err_N4, 'b-+',label='N-4')
loglog(H,err_RK4, 'g-o',label='RK41')
xlabel('$h$')
ylabel('$e$')
title("Schemas explicites")
legend()
grid(True)
```

```
subplot(1,3,2)
loglog(H,err_er, 'r-o',label='AM-0=EI')
loglog(H,err_CN, 'b-v',label='AM-1=CN')
loglog(H,err_AM2, 'm->',label='AM-2')
loglog(H,err_AM3, 'c-D',label='AM-3')
loglog(H,err_AM4, 'g-+',label='AM-4')
loglog(H,err_AM5, 'b->',label='AM-5')
loglog(H,err_BDF2, 'y-*',label='BDF-2')
loglog(H,err_BDF3, 'r-<',label='BDF-3')
loglog(H,err_MS2, 'm-o',label='MS-2')
xlabel('$h$')
ylabel('$e$')
title("Schemas implicites")
legend()
grid(True)

subplot(1,3,3)
loglog(H,err_em, 'c-o',label='EM')
loglog(H,err_heun, 'y->',label='Heun')
loglog(H,err_AM4AB2, 'r-<',label='AM4-AB2')
loglog(H,err_AM4AB3, 'b-v',label='AM4-AB3')
loglog(H,err_AM4AB4, 'm-*',label='AM4-AB4')
xlabel('$h$')
ylabel('$e$')
title("Schemas predicteur-correcteur")
legend()
grid(True)

show()
```



[ ]: